DEPARTMENT OF COMPUTER SCIENCE AND IT
NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

# COMPLEX COMPUTING PROBLEM (CCP) REPORT

Course Code: CT-351
Course Title: Knowledge Representation & Reasoning

| | |
|---|---|
| **Project Title:** | Smart Parking Access & Space Allocation Reasoning System (CarePark-KR) |
| **Selected Domain:** | Smart parking: hospital car park access and bay allocation (Car park P, Riverside General Hospital) |
| **Section:** | |
| **Submission Date:** | November 2026 |

## GROUP MEMBERS

| S. No. | Student Name | Roll Number | Assigned Contribution |
|---|---|---|---|
| 1 | Minhaj Khan | AI-23301 | Problem, knowledge acquisition, frames and script |
| 2 | Faraz Khan | AI-23047 | Semantic network, description logic, expressivity analysis |
| 3 | Raied Khan | AI-23046 | FOL, Horn knowledge base, forward chaining |
| 4 | Sabih Salam | AI-23040 | Backward chaining, website and 3D twin, testing, report |

**Instructor:** Miss Dure Shahwar

**CLO:** CLO-3 (Design/Analysis) **PLO:** PLO-10 (Lifelong Learning)

**Group:** ____ **Semester:** 7th

**Department of Computer Science and IT**
**Course Code: CT-351 | Course Title: Knowledge Representation & Reasoning**
**Complex Computing Problem (CCP): Grading Rubric**

| Student No. | Name | Roll No. |
|---|---|---|
| S1 | Minhaj Khan | AI-23301 |
| S2 | Faraz Khan | AI-23047 |
| S3 | Raied Khan | AI-23046 |
| S4 | Sabih Salam | AI-23040 |

| Criteria and scales | 0 (Unsatisfactory) | 1 (Developing) | 2 (Proficient) | S1 | S2 | S3 | S4 |
|---|---|---|---|---|---|---|---|
| Criterion 1: Problem Statement & Motivation | Objective of the project and domain requirements are not clear or described. | Some lack of clarity in objectives/purpose or domain justification. | The project's objectives and domain motivation are clearly stated and justified. | | | | |
| Criterion 2: Structural KR (Frames & Semantic Networks) | Model fails to represent necessary concepts, frames, slots, or semantic relationships. | Model includes frames and semantic networks but lacks complete slots, facets, or default values. | Model meets Proficient criteria with comprehensive, accurate frame structures and semantic networks. | | | | |
| Criterion 3: Formal Logic (FOL & Description Logic) | Logical representations contain major syntax/semantic errors or are missing. | FOL and DL formulas are present but contain minor syntax/quantifier errors. | Model represents domain constraints perfectly using correct FOL formulas and Description Logic axioms. | | | | |
| Criterion 4: Inference Engine (Horn Clauses & Chaining) | Rules are not converted to Horn clauses; failure to execute chaining. | Rules are in Horn clauses, but Forward/Backward chaining traces have minor logical gaps. | Horn clauses are correctly formulated with flawless, detailed Forward and Backward chaining traces. | | | | |
| Criterion 5: Team Participation & Individual Viva | The individual did not contribute to the project and failed to answer viva questions. | The individual contributed partially and demonstrated basic understanding during viva. | The individual contributed significantly and demonstrated mastery of KR concepts during viva. | | | | |
| Total Marks (Out of 10): | | | | | | | |

Teacher's Signature: _____ Date: _____

# REPORT CONTENTS

**1. INTRODUCTION** 4
1.1 Selected Domain and Background 4
1.2 Problem Statement and Objectives 4
**2. DOMAIN ANALYSIS** 5
2.1 Domain Description and Requirements 5
2.2 Entities, Relationships, and Challenges 7
**3. KNOWLEDGE REPRESENTATION** 10
3.1 Frames 10
3.2 Semantic Networks 16
**4. FORMAL LOGIC REPRESENTATION** 19
4.1 First-Order Logic (FOL) 19
4.2 Description Logic (DL) 20
**5. HORN CLAUSE KNOWLEDGE BASE** 24
5.1 Knowledge base 24
5.2 Facts 24
5.3 Rules 24
5.4 Horn clause conversion 25
5.5 Prolog form 26
**6. INFERENCE AND REASONING** 27
6.1 Forward Chaining 27
6.2 Backward Chaining 31
**7. IMPLEMENTATION** 35
7.1 Tools and Technologies 35
7.2 Code / Execution 35
**8. RESULTS AND DISCUSSION** 41
8.1 Results 41
8.2 Discussion 42
**9. CONCLUSION** 44
9.1 Conclusion and Future Improvements 44
**10. REFERENCES** 44
**11. TEAM PARTICIPATION** 44

# 1. INTRODUCTION

## 1.1 Selected Domain and Background

The selected domain is smart parking at a hospital: who may drive into Car park P at Riverside General Hospital, and which bay each vehicle should get. A hospital car park is harder than an office or campus car park because the stakes are uneven. An ambulance on a run must never wait, a dialysis patient must reach the unit on time, an on-call cardiologist must be close to the ward, and a visitor arriving outside visiting hours should be kept to the short-stay drop-off bays. The same car park also serves suppliers with delivery slots, staff with badges and shifts, and people with disability badges or wheelchair vans that cannot use the 2.1 m ramp.

Today these decisions are made by a gate guard with a printed list. Decisions differ from guard to guard, a refusal is rarely explained, and when the hospital declares a code red (mass-casualty mode) or the weather turns, the rules change in ways nobody has written down. Every one of these decisions is an inference from facts (who the driver is, what time it is, how tall the vehicle is, which bays are free) through policy rules to a conclusion. That is exactly the kind of problem knowledge representation and reasoning is built for.

## 1.2 Problem Statement and Objectives

Given the facts about people, vehicles, badges, shifts, appointments, delivery slots, levels, zones and bays for one day, decide for every arriving vehicle whether it is admitted, which bay it gets, how much it pays and what alerts follow, and explain each decision rule by rule. Every refusal must name the first condition that failed. The policy must be written once, as knowledge, and checked for consistency.

The objectives are:

1. Model the domain with class frames that carry slots, facets, defaults and demons (at least 25 class frames), a script for a parking visit, and a semantic network with is-a, has-a and part-of links.
2. Write the policy in first-order logic with explicit quantifiers, and the vocabulary as a description-logic TBox and ABox with subsumption, membership and disjointness axioms.
3. Convert the rules into Horn clauses, and run them with a forward chainer (data-driven) and a backward chainer (goal-driven) that print full traces.
4. Handle negation (not banned, no charger, not out of service) safely with stratified negation as failure.
5. Check the knowledge base with integrity constraints, and test the system with at least 20 scenarios covering normal days, policy changes, edge cases and bad data.
6. Show the reasoning in an interactive 3D digital twin of the car park, so an examiner can watch the day and question any decision.

The system covers one hospital car park with 4 levels, 13 zones, 29 bays and 8 hospital departments, and one full day (Tuesday 10 November 2026) with 15 arriving drivers and 3 cars parked overnight. Times are minutes after midnight (07:05 = 425) and dates are YYYYMMDD integers. It does not take payments, read real sensors or learn from data.

The solution has four layers (Figure 1). The knowledge layer holds 431 facts, 97 Horn rules, 13 integrity constraints and 22 scenarios. The reasoning layer has the forward chainer, the day-replay control layer, the backward chainer with a why-not explainer, and the constraint checker. The representation layer holds the frames, script, semantic network, FOL statements and DL ontology. The interface layer has a command-line program and a web control room with a 3D digital twin. An exporter writes the knowledge base to JSON, so the website runs exactly the same rules.

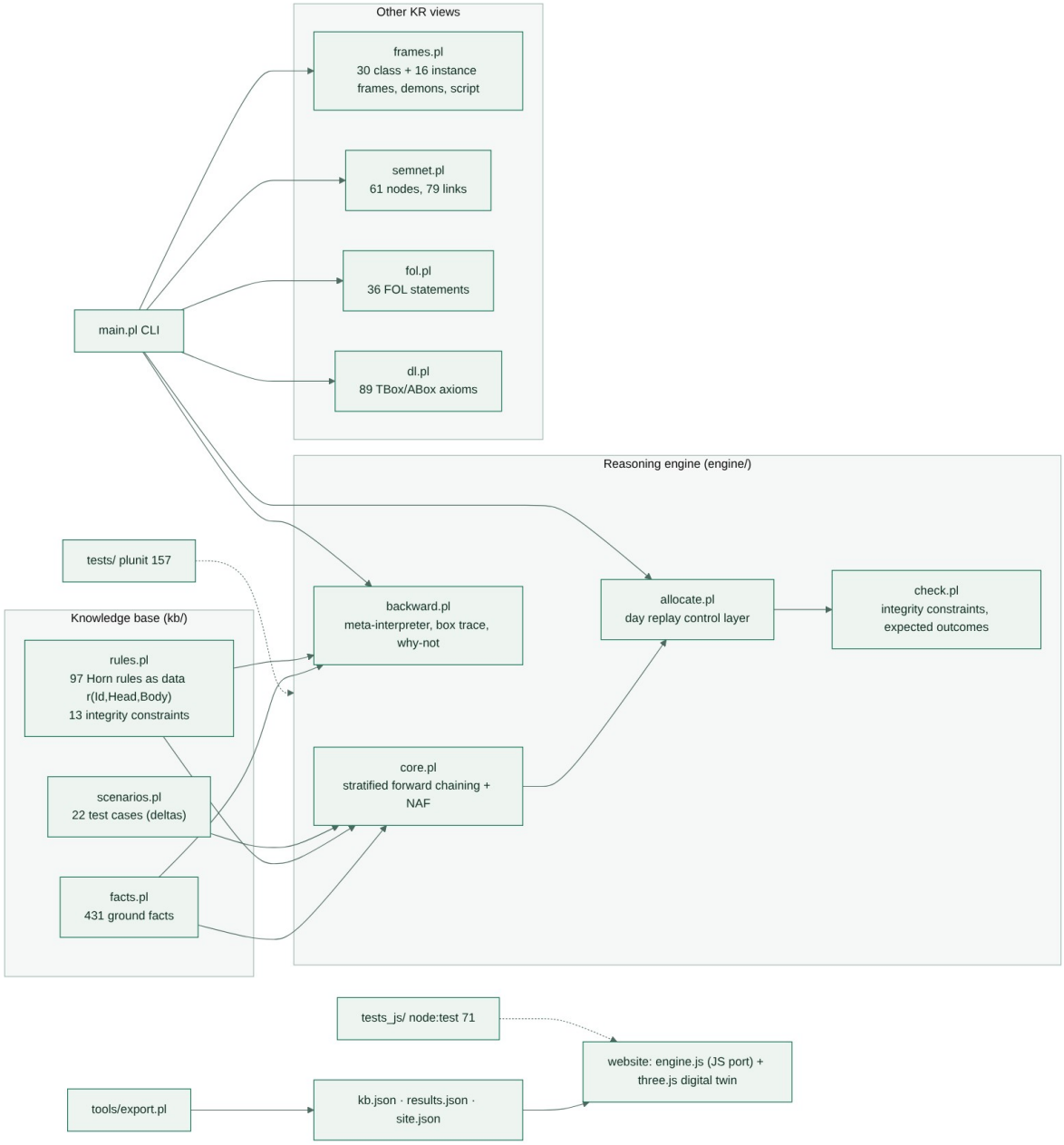



**Figure 1: System architecture of CarePark-KR**

# 2. DOMAIN ANALYSIS

## 2.1 Domain Description and Requirements

### 2.1.1 Hospital parking domain

Car park P has a ground level with a 4.0 m clearance and three upper levels (L1, L2 and an open roof) reached by an internal ramp with a 2.1 m clearance. The ground level holds the ambulance bays, drop-off bays, accessible bays, the loading bay, EV charger bays and motorcycle bays. L1 holds the on-call, staff, patient and maternity zones; L2 holds the visitor and family zones; the roof is overflow. Bay vs4 is out of service on the day.

**Table 1: Levels, zones and bays of Car park P**

| Level | Route clearance | Zones (kind) | Bays |
|---|---|---|---|
| Ground (g) | 400 cm | z_amb (ambulance), z_drop (drop-off), z_acc (accessible), z_load (loading), z_ev (charger), z_moto (motorcycle) | amb1–2, dr1–2, ac1–2, ld1, ev1–2, mc1–2 (11) |
| L1 | 210 cm | z_oncall, z_staff, z_patient, z_mat (maternity) | oc1–2, st1–4, pt1–3, mt1 (10) |

| Level | Route clearance | Zones (kind) | Bays |
|---|---|---|---|
| L2 | 210 cm | z_visit, z_family | vs1–4, fm1 (5; vs4 out of service) |
| Roof | 210 cm | z_roof (overflow, open air) | rf1–3 (3) |

A walking table gives the distance in metres from each of the 13 zones to each of the 8 departments (emergency, cardiology, oncology, maternity, OPD, dialysis, pharmacy, admin), 104 entries in all. The main scenario is one day. Fifteen drivers arrive between 06:40 and 15:05, and three cars parked overnight (Zubair's EV in ev2, Hamid and Lubna in st1 and st2) leave during the day.

**Table 2: Drivers on the main scenario day**

| Driver | Role | Going to | Vehicle | Arrives | What makes the case interesting |
|---|---|---|---|---|---|
| Saad | doctor | cardiology | ev_sa (EV, 22%) | 07:05 | on call, low battery |
| Mariam | nurse | maternity | mc_ma (motorcycle) | 06:40 | first arrival |
| Areeba | doctor | OPD | suv_ar (SUV) | 07:50 | normal shift |
| Imtiaz | patient | dialysis | car_im | 07:45 | appointment 08:00 |
| Junaid | supplier | pharmacy | van_ju (260 cm) | 08:40 | delivery slot 08:30–09:30 |
| Faisal | paramedic | emergency | amb_fa (ambulance) | 09:00 | ambulance on a run |
| Bushra | admin | admin | car_bu | 09:20 | badge expired 31 Oct |
| Noor | patient | oncology | wv_no (wheelchair van, 230 cm) | 09:45 | disability badge |
| Kashif | visitor | cardiology | car_ks | 10:15 | outside visiting hours |
| Sadia | visitor | cardiology | car_sd | 11:30 | with children |
| Rukhsana | visitor | maternity | car_ru (SUV) | 11:40 | expectant mother |
| Waqar | visitor | OPD | car_wq | 11:45 | security ban |
| Ghazala | visitor | emergency | car_gh | 12:40 | declared emergency |
| Tahir | patient | OPD | car_th | 14:00 | three hours late for 11:00 |
| Yasir | visitor | oncology | car_ya | 15:05 | plate read at 64% |

### 2.1.2 Knowledge acquisition

We collected the documents a hospital estates office would hold: the parking policy for staff, patients, visitors and suppliers; the ambulance bay rules; the disability and maternity bay rules; the delivery slot sheet; the visiting hours (11:00–13:00 and 17:00–20:00); the code red procedure; and the drawings of the car park with its clearances. We then interviewed the process the way a guard runs it, asking "what happens if…" for each kind of driver. Each answer was written as an if-then sentence, for example "a staff member whose badge is in date and who is not banned is cleared". Nouns in these sentences became classes and frame slots, verbs became relations, and repeated patterns ("doctors and nurses are clinicians") became superclasses.

Each requirement then became a test scenario with an expected answer, and we kept refining the knowledge base until every scenario passed. Two errors found this way: the first draft let Noor's 230 cm wheelchair van be sent to the L1 patient zone (fixed by the route clearance check in rule f1), and the first draft charged the roof the same in rain as in dry weather (fixed by k8).

### 2.1.3 Domain requirements

The rules below came out of knowledge acquisition. The rule numbers in brackets show where each one is implemented.

1. An ambulance on an emergency run, or a vehicle whose driver declares an emergency, is admitted at once (a5, a6) and served first (p1, p2).
2. A plate must be read with at least 90% confidence to identify the vehicle (s1); otherwise it is refused and sent to a manual check (w9).

3. Staff are cleared only with a badge that is in date (s4, a1) and only while on shift, from 30 minutes before the shift start to its end (t1).
4. Patients are cleared (a2) and get the patient zone only inside their appointment window, 60 minutes before to 30 minutes after (t2, e6).
5. Visitors are cleared (a3); they get the visitor and family zones only in visiting hours (t3, e11, e12), otherwise only the drop-off bays.
6. Suppliers are cleared only inside their delivery slot (t4, a4) and use the loading bay (e7).
7. Anyone on the security ban list is never cleared (s5, NAF in a1–a4) and raises a security alert (w10).
8. A vehicle fits a bay only if its class can use the bay type and its height is within the route clearance of the bay's level (f1).
9. Accessible bays are for disability badge holders (e3), chargers for EVs (e5), the on-call zone for on-call doctors (e8), the maternity zone for expectant mothers (e10).
10. The bay with the lowest cost is chosen: walking metres to the destination plus the zone penalty (drop-off 300 m, roof 60 m, roof in rain 360 m), plus 120 m for a low-battery EV in a bay without a charger, or 40 m for a charged EV taking a charger (k1–k11).
11. Vehicles arriving in the same minute are served by priority: emergency 0, on-call doctor 1, patient in window or expectant mother 2, staff and suppliers 3, visitors 4, everyone else 5 (p1–p10).
12. If no candidate bay is free, the vehicle waits and is placed when a bay is released (control layer).
13. In code red, visitors lose the visitor, family and roof zones (e11, e12, e15).
14. After allocation: a non-emergency vehicle in an ambulance bay is reported, and if it is unregistered a tow is requested (w3, w4); drop-off stays over 15 minutes (30 for emergencies) are flagged (w6–w8); a full EV left on a charger is a charger hog (w5); the maternity ward is told an expectant mother has arrived (w11); the trauma team is alerted for a declared emergency (w13).
15. Fees: staff on shift, emergencies and suppliers pay nothing; patients get two hours free then PKR 50 per started hour; visitors pay PKR 100 for the first hour and PKR 50 per extra hour (m1–m7).

### 2.1.4 Domain constraints

Some requirements are constraints on the result rather than rules that derive facts: no bay holds two vehicles at the same time, no vehicle holds two bays, nothing taller than the clearance goes above ground, and every vehicle has exactly one class and one driver. These are integrity constraints IC1–IC13 (section 5.3) and DL restrictions (section 4.2). The policy also has exceptions the rules must handle: an emergency overrides every other check, a code red overrides visiting hours, and an EV with a low battery prefers a charger even when another bay is closer.

## 2.2 Entities, Relationships, and Challenges

### 2.2.1 Domain entities

**Table 3: Domain entities**

| Entity | Description | Important attributes (frame slots) |
|---|---|---|
| Person | Anyone who may drive in | name, role, destination, drives, banned, cleared |
| Staff (Doctor, Nurse, Paramedic, AdminOfficer) | Hospital employees with badges and shifts | badge_expiry, shift_start, shift_end, badge_valid, on_call |
| Patient | Person with an appointment | appointment_time, appointment_dept, fee_band |
| Visitor | Person visiting a ward | ward, fee_band |
| Supplier | Delivery driver with a slot | delivery_from, delivery_to |
| BadgeHolder, ExpectantMother | Roles that add needs to a person | disability_badge, preferred_zone, notify_ward |
| Vehicle (Car, SUV, ElectricCar, Van, WheelchairVan, Ambulance, Motorcycle) | What arrives at the gate | plate, class, height_cm, anpr_confidence, arrival, stay_min, battery_pct, occupies |
| Level | Floor of the car park | floor, clearance_cm |
| Zone | Group of bays with one purpose | kind, level, bays, penalty_m |

| Entity | Description | Important attributes (frame slots) |
|---|---|---|
| Bay (AmbulanceBay, AccessibleBay, ChargerBay, LoadingBay, DropOffBay) | One parking space | bay_type, zone, status, width_m, sensor, charger_kw, max_stay_min |
| Department | Hospital destination | entrance, walking distance from each zone |
| Appointment | A patient's booking | patient, time, window |

### 2.2.2 Domain relationships

People are arranged in a class hierarchy by is-a (a Doctor is-a Staff, Staff is-a Person); vehicles likewise (a WheelchairVan is-a Van is-a Vehicle). The facility is arranged by part-of: a bay is part of a zone, a zone is part of a level, a level and the gate are part of the car park, and the car park and the departments are part of the hospital. Has-a links give components: a vehicle has a number plate, an electric car has a battery, a charger bay has a charger, the gate has an ANPR camera and a barrier. Named relations connect individuals: a person drives a vehicle, a staff member works in a department, a patient has an appointment at a department, a visitor visits a ward, a vehicle occupies a bay, and a zone has a walking distance to each department.

Two individuals inherit from two classes at once: Noor is both a Patient and a BadgeHolder, and Rukhsana is both a Visitor and an ExpectantMother. This is why the frame system supports multiple inheritance (section 3.1).

### 2.2.3 Challenges in representation and reasoning

- **Long chains.** "Rukhsana gets mt1" depends on about 20 rule firings: her role, the visiting window, her expectant status, the maternity zone, the ramp height, the walking table and the bays already taken.
- **Negation.** Many rules depend on something being absent: not banned, not out of service, no charger, not an emergency. Negation must be evaluated only after the predicate it negates is complete, so the rules are stratified.
- **Defaults and exceptions.** Most vehicles are 150 cm tall, but a wheelchair van defaults to 230 cm and an ambulance to 280 cm. Most bays are 2.5 m wide, but an accessible bay is 3.6 m.
- **Context changes the rules.** Code red closes visitor zones; rain makes the open roof expensive; the time of day decides visiting and appointment windows.
- **Competition for bays.** The answer for one vehicle depends on who arrived earlier and who has higher priority, so the decision is not a pure logical consequence. A control layer replays the day in time order.
- **Explanation.** A bare "no" does not help a driver. Every refusal must name the first failing condition.
- **Bad data.** A car recorded with two classes, a doctor also listed as a visitor, or a badge that expires before it was issued must be caught before any decision is trusted.

### 2.2.4 Limitations of traditional database systems

A relational database stores every fact well, but it cannot do the reasoning the domain needs:

**Table 4: Database vs knowledge-based approach**

| Need | Relational database | CarePark-KR |
|---|---|---|
| Derive new facts (staff → person, ambulance on a run → emergency) | Views written by hand; recursive chains are awkward | 97 rules chained to a fixed point |
| Negation with a clear meaning | NOT EXISTS with no stratification check | Negation as failure in 3 checked strata |
| Explain a refusal | Returns no rows | Why-not tree down to the failed fact or comparison |
| Inheritance and defaults | Not supported | Frames with defaults and if-needed demons |
| Consistency of the vocabulary | Constraints on single tables | DL disjointness and ∀-restrictions, 13 integrity constraints |
| Change the policy | Code and schema migrations | Edit the rules file; rules are data |

# 3. KNOWLEDGE REPRESENTATION

Five paradigms describe the same knowledge base, each for what it does best. The facts and Horn rules are the single source; the other views are built over the same facts, and a test checks that they agree.

**Table 5: Role of each paradigm in the project**

| Paradigm | Purpose in this project | Example |
|---|---|---|
| Frames and script | Concepts with slots, facets, inherited defaults and demons; the stages of a parking visit | WheelchairVan: height_cm default 230; Staff: badge_valid if-needed |
| Semantic network | Concepts and individuals with typed links; inheritance by graph walk | amb1 part_of z_amb part_of g |
| FOL | The exact policy with quantifiers, including what Horn clauses cannot say | ∀p (Banned(p) → ¬Cleared(p)) |
| Description logic | Class hierarchy, disjointness, restrictions, realisation, consistency | AmbulanceBay ⊑ ∀occupiedBy.EmergencyVehicle |
| Horn clauses | The executable policy, run forward and backward | admitted(V) :- arrival(V,_), identified(V), drives(P,V), cleared(P). |

## 3.1 Frames

### 3.1.1 Frame architecture

A frame is `frame(Name, Kind, Parents, Slots)`. Kind is class or instance, and Parents is a list, so a frame can inherit from more than one class. Each slot holds a list of facets. The system has 30 class frames and 16 instance frames (engine/frames.pl).

**Table 6: Facets used in the frames**

| Facet | Meaning | Example |
|---|---|---|
| value | A fixed value, inherited by every descendant | AmbulanceBay: bay_type value amb |
| default | Used only when nothing more specific exists | Vehicle: height_cm default 150 |
| type | Kind of filler allowed | Person: banned type boolean |
| range | Allowed numeric range | Vehicle: height_cm range 80–400 |
| card | How many fillers | Person: drives card 0..2; Zone: bays card 1..* |
| if_needed | Demon run when the slot is read and has no value | Staff: badge_valid if_needed d_badge_valid |
| if_added | Demon run when a value is put | Vehicle: occupies if_added d_park |

Slot lookup follows a fixed order: (1) a local value on the frame itself; (2) an if-needed demon found on the frame or an ancestor; (3) a value facet inherited along the is-a chain; (4) a default facet. The chain is found breadth-first, so for Noor it is noor → Patient → BadgeHolder → Person → Thing, and a nearer class always wins over a farther one.

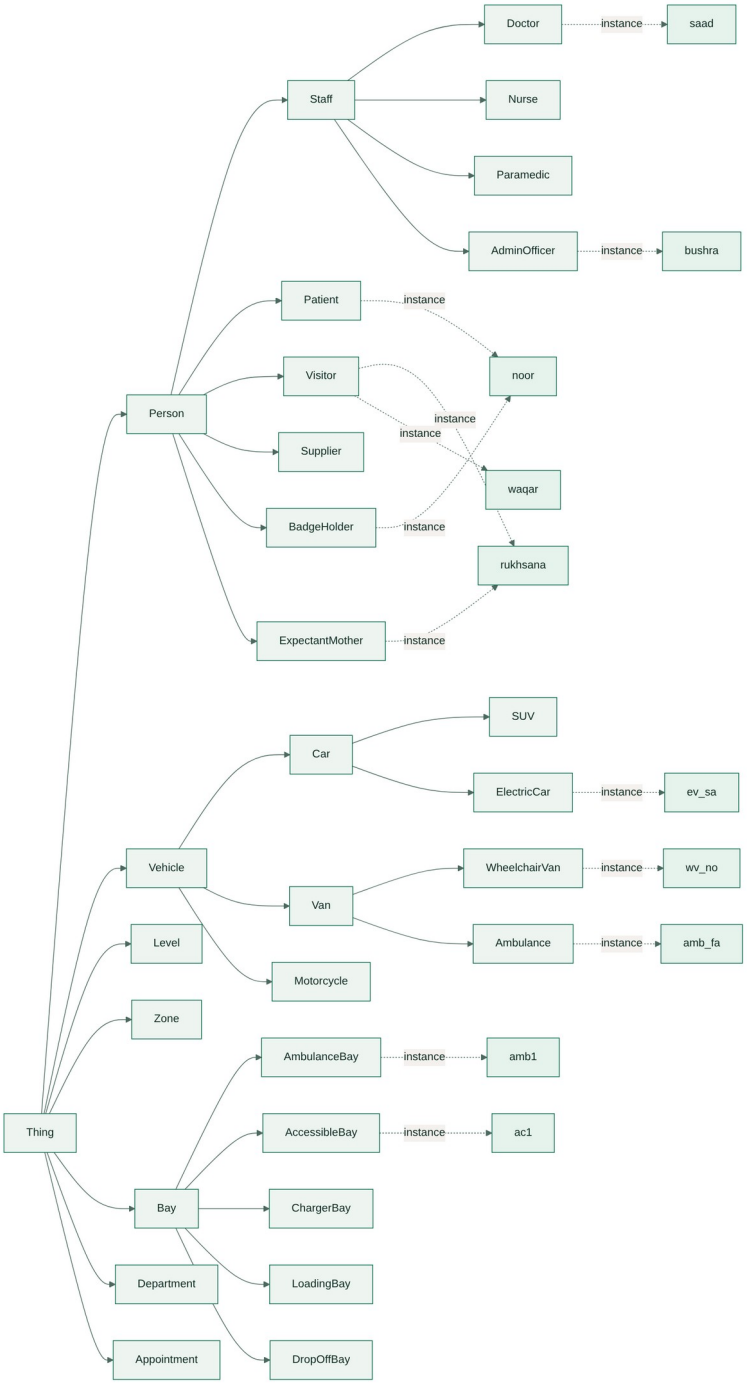



**Figure 2: Frame hierarchy with multiple inheritance (dotted arrows: instance frames)**

### 3.1.2 Frame: Person

Person is the root of the people hierarchy. Its six slots are inherited by every role.

**Table 7: Frame Person**

| Slot | Facets | Inherited by |
|---|---|---|
| `name` | type text | all people |
| `role` | type role | all people; fixed by each subclass |
| `destination` | type department | all people |
| `drives` | type Vehicle, card 0..2 | all people |
| `banned` | type boolean, default false | all people |
| `cleared` | type boolean, if_needed d_cleared | all people |

### 3.1.3 Frame: Staff

Staff is-a Person. Doctor, Nurse, Paramedic and AdminOfficer are its subclasses; Doctor adds on_call (default false), Paramedic fixes destination to emergency, AdminOfficer fixes destination to admin.

**Table 8: Frame Staff**

| Slot | Facets | Inherited by |
|---|---|---|
| badge_expiry | type date | all staff |
| shift_start, shift_end | type minute | all staff |
| badge_valid | type boolean, if_needed d_badge_valid | all staff |
| fee_band | default free | all staff |
| role | value doctor / nurse / paramedic / admin | set by each subclass |

### 3.1.4 Frame: Patient

Patient is-a Person. Its window and fee band are the defaults the policy applies to every patient.

**Table 9: Frame Patient**

| Slot | Facets | Inherited by |
|---|---|---|
| appointment_time | type minute | patients |
| appointment_dept | type department | patients |
| role | value patient | patients |
| fee_band | default first 2 h free, then PKR 50/h | patients |

### 3.1.5 Frame: Visitor, Supplier, BadgeHolder, ExpectantMother

These four are siblings under Person. BadgeHolder and ExpectantMother are mixed into other roles by multiple inheritance.

**Table 10: Frame Visitor, Supplier, BadgeHolder, ExpectantMother**

| Slot | Facets | Inherited by |
|---|---|---|
| ward (Visitor) | type department | visitors |
| fee_band (Visitor) | default PKR 100 first hour, PKR 50/h after | visitors |
| delivery_from, delivery_to (Supplier) | type minute | suppliers |
| disability_badge (BadgeHolder) | value true | badge holders |
| preferred_zone (BadgeHolder) | default accessible | badge holders |
| preferred_zone (ExpectantMother) | default maternity | expectant mothers |
| notify_ward (ExpectantMother) | default true | expectant mothers |

### 3.1.6 Frame: Vehicle

Vehicle is the root of the vehicle hierarchy. Car (default class car, 4 wheels), SUV (185 cm), ElectricCar (battery, needs_charge demon), Van (260 cm, max_level demon), WheelchairVan (230 cm, rear ramp), Ambulance (280 cm, siren) and Motorcycle (2 wheels, 120 cm) refine it.

**Table 11: Frame Vehicle**

| Slot | Facets | Inherited by |
|---|---|---|
| plate | type text | all vehicles |

| Slot | Facets | Inherited by |
|---|---|---|
| class | type vclass; value set by subclass | all vehicles |
| height_cm | type integer, range 80–400, default 150 | overridden by SUV, Van, WheelchairVan, Ambulance, Motorcycle |
| anpr_confidence | type percent | all vehicles |
| arrival, stay_min | type minute | all vehicles |
| identified | type boolean, if_needed d_identified | all vehicles |
| occupies | type Bay, card 0..1, if_added d_park | all vehicles |
| battery_pct, needs_charge (ElectricCar) | type percent; if_needed d_needs_charge | electric cars |
| max_level (Van) | if_needed d_max_level | vans and their subclasses |

### 3.1.7 Frame: Bay and its subclasses

Bay holds the defaults for an ordinary space; each subclass fixes its type and size.

**Table 12: Frame Bay and its subclasses**

| Slot | Facets | Inherited by |
|---|---|---|
| bay_type | default std; value amb / acc / ev / load / drop in subclasses | all bays |
| zone | type Zone | all bays |
| status | default vacant | all bays |
| width_m | default 2.5; value 3.5 (ambulance), 3.6 (accessible), 4.0 (loading) | all bays |
| sensor | default ultrasonic | all bays |
| keep_clear (AmbulanceBay) | value true | ambulance bays |
| charger_kw (ChargerBay) | default 22 | charger bays |
| max_stay_min (DropOffBay) | default 15 | drop-off bays |

### 3.1.8 Frame: Level, Zone, Department, Appointment

The facility and booking frames.

**Table 13: Frame Level, Zone, Department, Appointment**

| Slot | Facets | Inherited by |
|---|---|---|
| floor (Level) | type integer | levels |
| clearance_cm (Level) | type integer, default 210 | levels; ground overrides with 400 |
| kind, level (Zone) | type zone_kind; type Level | zones |
| bays (Zone) | card 1..* | zones |
| penalty_m (Zone) | default 0 | zones |
| entrance, walk_from_zone_m (Department) | type text; type table | departments |
| window (Appointment) | default -60 / +30 min | appointments |

### 3.1.9 Instance frames and demons

Instance frames hold the facts of the day. The first table shows Noor, who inherits from two classes; the second shows Bushra, whose refusal comes from a demon.

**Table 14: Instance frame noor (multiple inheritance)**

| Slot | Value | Where it came from |
|---|---|---|
| appointment_dept | oncology | local value |
| appointment_time | 600 (10:00) | local value |
| drives | wv_no | local value |
| role | patient | inherited from Patient |
| disability_badge | true | inherited from BadgeHolder |
| preferred_zone | accessible | default of BadgeHolder |
| fee_band | first 2 h free, then PKR 50/h | default of Patient |
| banned | false | default of Person |
| cleared | true | if-needed demon of Person |

**Table 15: Instance frame bushra (refusal from a demon)**

| Slot | Value | Where it came from |
|---|---|---|
| badge_expiry | 20261031 | local value |
| badge_valid | false | if-needed demon of Staff: 20261031 < today 20261110 |
| cleared | false | if-needed demon of Person: badge invalid |
| destination | admin | inherited from AdminOfficer |
| fee_band | free | default of Staff |

**Table 16: Demons**

| Demon | Slot | Procedure |
|---|---|---|
| d_badge_valid | badge_valid | true if badge_expiry ≥ today |
| d_cleared | cleared | false if banned or badge invalid, else true |
| d_identified | identified | true if anpr_confidence ≥ 90 |
| d_needs_charge | needs_charge | true if battery_pct < 30 |
| d_max_level | max_level | ground_only if height_cm > 210, else any |
| d_park (if-added) | occupies | records occupied(Bay, Vehicle) when a vehicle is put in a bay |

For example, `put(ev_sa, occupies, ev1)` fires d_park, which adds `occupied(ev1, ev_sa)`; reading `ev_sa.needs_charge` fires d_needs_charge, which returns true because the battery is at 22%.

### 3.1.10 Script / event sequence

The PARKING-VISIT script orders a visit into 10 scenes. Each scene has an actor, what happens, and the rules that carry it out (Figure 3). Scene 3 is a branch: an emergency skips scenes 4 and 5. Scene 6 is a second branch: a vehicle that is not cleared stops at the barrier.

**Table 17: Script PARKING-VISIT**

| # | Scene | Actor | What happens | Rules |
|---|---|---|---|---|
| 1 | Approach | Driver | Vehicle joins the queue at gate G1; arrival time recorded | arrival |
| 2 | Plate read | ANPR camera | Plate read with a confidence score; under 90% a guard checks it | s1, w9 |
| 3 | Emergency check | Gate controller | Ambulance on a run or declared emergency skips every other check | a5, a6, p1, p2 |
| 4 | Identification | Gate controller | Look up the driver's role, badge, appointment, delivery slot or visit | h1–h10, t1–t4 |
| 5 | Clearance | Security office | Badge in date, not banned, supplier inside its slot | s4, s5, a1–a4 |

| # | Scene | Actor | What happens | Rules |
|---|---|---|---|---|
| 6 | Entry | Barrier | Opens for an admitted vehicle, otherwise shows the first failing reason | a7 |
| 7 | Bay choice | Allocation engine | Fit and height, entitlement, cost, priority; one bay chosen | f1, e1–e15, c1, k9 |
| 8 | Guidance | Display board | Bay and route shown; ward or trauma team notified | w11–w13 |
| 9 | Parking | Occupancy sensor | Sensor confirms the vehicle; status := occupied | w2 |
| 10 | Stay and exit | Monitoring | Drop-off limits, charger use and blocked bays watched; fee charged | w3, w5, w8, m1–m7 |

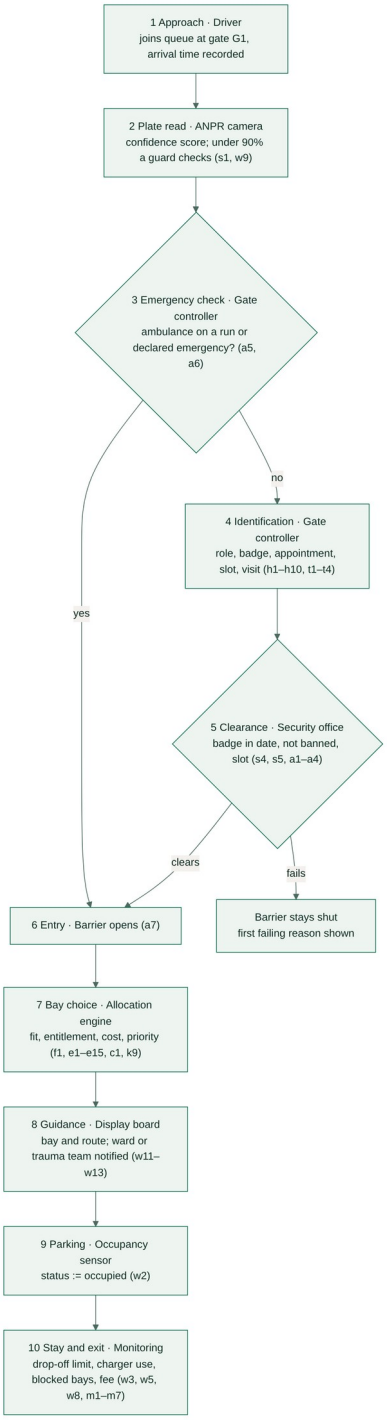



**Figure 3: The PARKING-VISIT script**

## 3.2 Semantic Networks

### 3.2.1 Semantic network design

The semantic network has 61 nodes and 79 labelled links, stored as `link(From, Relation, To)` (engine/semnet.pl). Class nodes (Person, Bay, Zone…), component nodes (NumberPlate, Battery, ANPRCamera…) and individual nodes (noor, saad, amb1…) share one graph, so inheritance and "how is X related to Y" are both graph walks.

### 3.2.2 Relationship types

**Table 18: Relationship types in the semantic network**

| Relation | Links | Meaning | Example |
|---|---|---|---|
| is_a | 25 | subclass to superclass | WheelchairVan is_a Van; OnCallDoctor is_a Doctor |
| instance_of | 16 | individual to class | noor instance_of Patient, noor instance_of BadgeHolder |
| has_a | 10 | whole to component | Gate has_a ANPRCamera; ElectricCar has_a Battery |
| part_of | 9 | part to whole | Bay part_of Zone part_of Level part_of CarPark part_of Hospital |
| drives | 5 | person to vehicle | faisal drives amb_fa |
| works_in, visits, has, at | 8 | people to departments and bookings | rukhsana visits maternity; appt_noor at oncology |
| walk_to, occupies, reads, shows, delivers_to, limited_by | 6 | other named relations | Zone walk_to Department; Ramp limited_by Clearance |

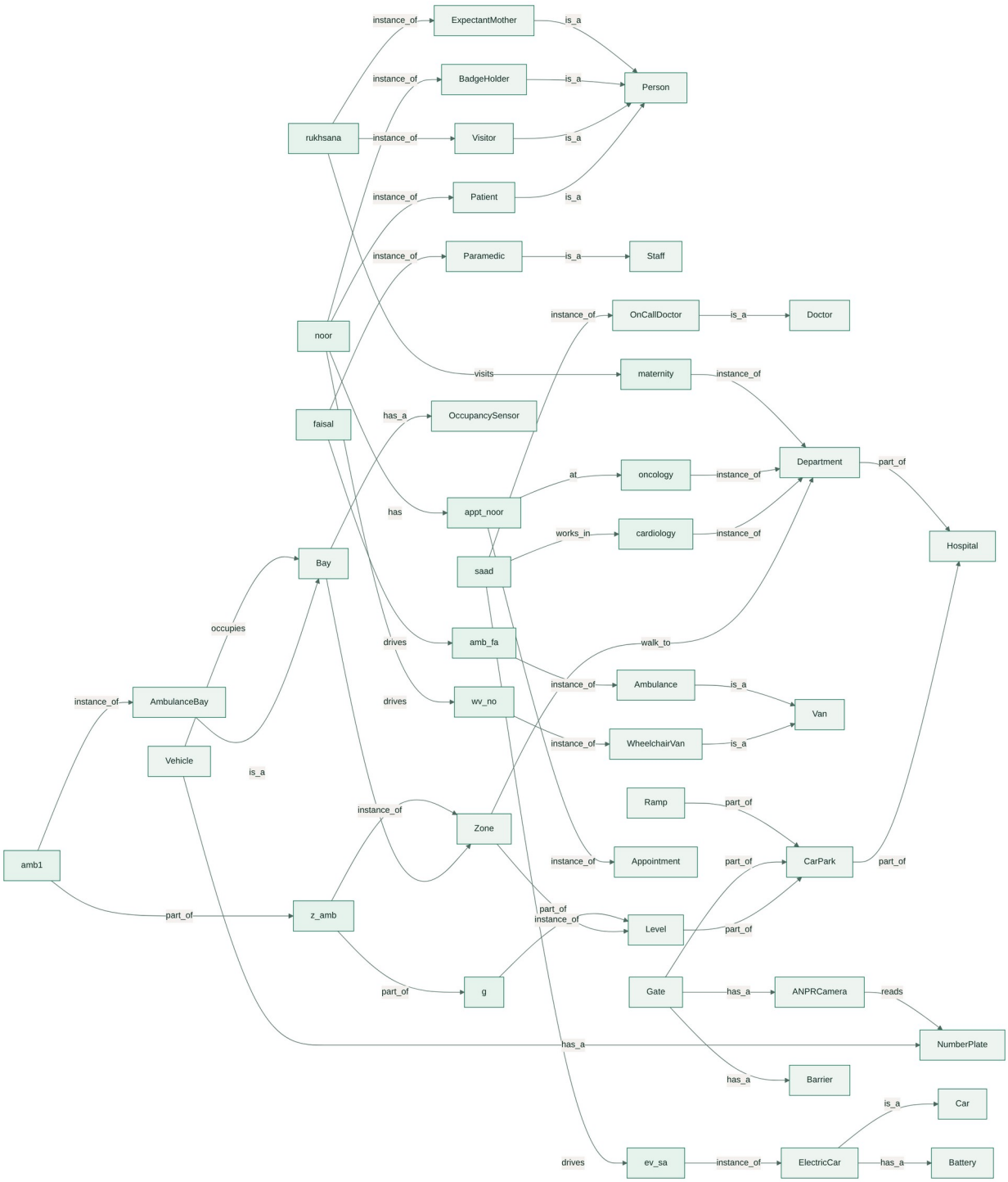



**Figure 4: Semantic network (individuals have lower-case names)**

### 3.2.3 Semantic network walkthrough

Four queries show how the network is used (output of `swipl main.pl net`):

- **Classes of ev_sa:** follow instance_of to ElectricCar, then is_a to Car and Vehicle. Result: ElectricCar, Car, Vehicle.
- **Inherited parts of ev_sa:** collect has_a links from each class. Result: Battery (from ElectricCar) and NumberPlate (from Vehicle).
- **Part-of chain of amb1:** follow part_of upward. Result: z_amb, then g (the ground level).
- **Path from noor to oncology:** breadth-first search over all links. Result: noor has appt_noor, appt_noor at oncology.

The website draws all 79 links as a force-directed graph; clicking a node lights up every class and part it inherits.

# 4. FORMAL LOGIC REPRESENTATION

## 4.1 First-Order Logic (FOL)

### 4.1.1 Predicates and constants

Constants name individuals (saad, noor, ev_sa, amb1, z_mat, l1, cardiology), roles, classes and numbers. Times are minutes after midnight and dates are YYYYMMDD integers, so comparisons are arithmetic. The knowledge base uses 42 base predicates; the main ones are listed below.

**Table 19: Main predicates**

| Predicate | Meaning |
|---|---|
| Doctor(x), Nurse(x), Admin(x), Paramedic(x), Patient(x), Visitor(x), Supplier(x) | role of person x |
| Drives(p,v), Class(v,c), Height(v,h), Anpr(v,c), Arrival(v,t), Stay(v,m) | vehicle facts |
| Badge(p,i,e), Shift(p,s,e), Appt(p,d,a), Slot(p,a,b), Banned(p), OnCall(p) | permissions and schedule |
| Type(b,t), InZone(b,z), Kind(z,k), LevelOf(z,l), Clear(l,c), Walk(z,d,m), OutOfService(b) | facility facts |
| Identified(v), Cleared(p), Admitted(v), Fits(v,b), Entitled(v,z), Candidate(v,b), Cost(v,b,c), Priority(v,n), Alloc(v,b) | derived predicates |

### 4.1.2 Domain facts

Examples of ground facts (the full set has 431):

```
Doctor(saad)   OnCall(saad)   Drives(saad, ev_sa)   Class(ev_sa, ev)   Battery(ev_sa, 22)
Arrival(ev_sa, 425)   Anpr(ev_sa, 98)   Shift(saad, 420, 1140)   Destination(saad, cardiology)
Patient(noor)   DisabilityBadge(noor)   Appt(noor, oncology, 600)   Height(wv_no, 230)
Admin(bushra)   Badge(bushra, 20241101, 20261031)   Today(20261110)   Visitor(waqar)   Banned(waqar)
Type(amb1, amb)   InZone(amb1, z_amb)   LevelOf(z_amb, g)   Clear(g, 400)   Clear(l1, 210)
Walk(z_mat, maternity, 35)   VisitingWindow(660, 780)   VisitingWindow(1020, 1200)   OutOfService(vs4)
```

### 4.1.3 FOL rules

The FOL layer has 36 statements (engine/fol.pl). 15 translate directly into Horn rules; the other 21 need disjunction in the head, existential or uniqueness quantifiers, or a negative conclusion, so they are checked on the finished model instead (section 4.1.5).

**Table 20: FOL statements (selection)**

| Id | Statement | Form |
|---|---|---|
| f1 | ∀x (Doctor(x) ∨ Nurse(x) ∨ Admin(x) ∨ Paramedic(x) → Staff(x)) | Horn: h1–h4 |
| f3 | ∀x (Person(x) → Staff(x) ∨ Patient(x) ∨ Visitor(x) ∨ Supplier(x)) | beyond Horn |
| f4 | ∀v (Vehicle(v) → ∃!c Class(v,c)) | beyond Horn: IC6 |
| f6 | ∀v (Identified(v) ↔ ∃c (Anpr(v,c) ∧ c ≥ 90)) | Horn: s1 |
| f7 | ∀p (BadgeValid(p) ↔ ∃i ∃e ∃t (Badge(p,i,e) ∧ Today(t) ∧ i ≤ t ≤ e)) | Horn: s4 |
| f8 | ∀p (OnShift(p) ↔ ∃s ∃e ∃v ∃t (Shift(p,s,e) ∧ Drives(p,v) ∧ Arrival(v,t) ∧ s − 30 ≤ t ≤ e)) | Horn: t1 |
| f9 | ∀p (InWindow(p) ↔ ∃d ∃a ∃v ∃t (Appt(p,d,a) ∧ Drives(p,v) ∧ Arrival(v,t) ∧ a − 60 ≤ t ≤ a + 30)) | Horn: t2 |
| f11 | ∀p (Banned(p) → ¬Cleared(p)) | beyond Horn: NAF in a1–a4 |
| f12 | ∀p (Staff(p) ∧ BadgeValid(p) ∧ ¬Banned(p) → Cleared(p)) | Horn with NAF: a1 |
| f13 | ∀v (EmergencyVehicle(v) ∨ DeclaredEmergency(v) → Admitted(v)) | Horn: a5, a6 |
| f14 | ∀v ∀p (Arrival(v) ∧ Identified(v) ∧ Drives(p,v) ∧ Cleared(p) → Admitted(v)) | Horn: a7 |
| f16 | ∀v ∀b (Fits(v,b) ↔ ∃c ∃t ∃h ∃z ∃l ∃k (Class(v,c) ∧ Type(b,t) ∧ ClassFits(c,t) ∧ Height(v,h) ∧ InZone(b,z) ∧ LevelOf(z,l) ∧ Clear(l,k) ∧ h ≤ k)) | Horn: f1 |
| f17 | ∀v ∀b ∀h (Alloc(v,b) ∧ Height(v,h) ∧ h > 210 → ∃z (InZone(b,z) ∧ LevelOf(z,g))) | beyond Horn: IC3 |

| Id | Statement | Form |
|---|---|---|
| f19 | ∀v ∀z (CodeRed ∧ Visitor(v) ∧ Kind(z,visitor) → ¬Entitled(v,z)) | beyond Horn: NAF in e11 |
| f20 | ∀v ∀b (Candidate(v,b) ↔ Admitted(v) ∧ Fits(v,b) ∧ ∃z (InZone(b,z) ∧ Entitled(v,z)) ∧ Usable(b)) | Horn: c1 |
| f22 | ∀v (Admitted(v) ∧ ∃b FreeCandidate(v,b) → ∃!b Alloc(v,b)) | beyond Horn: control layer |
| f23 | ∀b ∀$v_1$ ∀$v_2$ (Alloc($v_1$,b,[$s_1$,$e_1$]) ∧ Alloc($v_2$,b,[$s_2$,$e_2$]) ∧ $v_1 \neq v_2$ → $e_1 \leq s_2$ ∨ $e_2 \leq s_1$) | beyond Horn: IC1 |
| f24 | ∀v ∀b ∀t (Alloc(v,b,t) → ¬∃b′ (Free(b′,t) ∧ Candidate(v,b′) ∧ Cost(v,b′) < Cost(v,b))) | beyond Horn: control layer |
| f26 | ∀v ∀b (Occupies(v,b) ∧ Type(b,amb) → EmergencyVehicle(v)) | beyond Horn: alert w3 |
| f31 | ∀v (Patient(v) ∧ Stay(v,s) ∧ s > 120 → Fee(v, 50·⌈(s−120)/60⌉)) | Horn: m1–m7 |
| f35 | ∀x ¬(Staff(x) ∧ Visitor(x)) | beyond Horn: IC13 |
| f36 | ∃b (Type(b,amb) ∧ ¬OutOfService(b)) | beyond Horn: model check |

### 4.1.4 Quantifier usage

- **Universal (∀)** states a rule for every individual of a kind: f1 for every doctor, nurse, admin officer and paramedic; f14 for every arriving vehicle.
- **Existential (∃) in the body** introduces a value the rule needs but does not return: in f6 the confidence c, in f8 the shift times s and e and the arrival time t. In Horn form these become variables that appear only in the body.
- **Existential in the head** claims something exists: f36 says some ambulance bay is in service; f27 says the driver of a vehicle in an accessible bay holds a badge. Horn clauses cannot conclude existence, so these are checked.
- **Uniqueness (∃!)** combines existence with equality: f4 (exactly one class) and f5 (exactly one driver). The ≤ 1 part becomes integrity constraints IC5 and IC6.
- **Biconditional (↔)** defines a predicate completely: f6, f7, f16, f20. The ← direction is the Horn rule; the → direction holds under the closed-world assumption (Clark completion).
- **Nested negated existential** in f24 states optimality: there is no free candidate bay that is cheaper. It is enforced by the control layer and confirmed by a model check.

### 4.1.5 Consistency verification

Each of the 21 statements beyond Horn has a check goal that is run on the finished model of the day. On the baseline day all 21 hold. In the bad-data scenarios the checks fail as intended: tc20 breaks f4 (a car with two classes), tc21 breaks f35 (a doctor also recorded as a visitor) and tc22 breaks f33's companion constraint IC9 (a badge that expires before it is issued).

## 4.2 Description Logic (DL)

The ontology (engine/dl.pl) has 89 TBox axioms: 38 atomic subsumptions, 27 disjointness axioms and 24 definitions and role restrictions. The ABox is built from the facts of the current day, so the reasoner checks the same world the rules decide over.

### 4.2.1 DL concepts

Atomic concepts: Person, Staff, Doctor, Nurse, Paramedic, AdminOfficer, Patient, Visitor, Supplier, BadgeHolder, ExpectantMother, Vehicle, Car, SUV, ElectricCar, PetrolCar, Van, WheelchairVan, Ambulance, Motorcycle, Bay and its seven subclasses, Zone, GroundZone, UpperZone, Level, Department. Defined concepts: Clinician, OnCallDoctor, OnShiftStaff, ClearedPerson, BannedPerson, EmergencyVehicle, AdmittedVehicle, LowBatteryEV, TallVehicle, UnregisteredVehicle.

### 4.2.2 DL roles

**Table 21: DL roles**

| Role | Domain → Range | Notes |
|---|---|---|
| drives / drivenBy | Person → Vehicle | drivenBy ≡ drives⁻ |
| parkedIn / occupiedBy | Vehicle → Bay | occupiedBy ≡ parkedIn⁻ |
| inZone | Bay → Zone | functional (≤ 1) |
| onLevel | Zone → Level | functional |
| badge, shift, slot | Person → Badge / Shift / Slot | used in ClearedPerson and OnShiftStaff |
| onCall, onRun, declared | Person / Vehicle → {true} | nominals |
| battery, height, clearance | Vehicle / Level → integer | concrete domains with < 30, > 210, ≤ 210 |

### 4.2.3 Subsumption axioms

```
Staff ⊑ Person       Patient ⊑ Person       Visitor ⊑ Person       Supplier ⊑ Person
BadgeHolder ⊑ Person ExpectantMother ⊑ Person Doctor ⊑ Staff        Nurse ⊑ Staff
Paramedic ⊑ Staff    AdminOfficer ⊑ Staff   OnCallDoctor ⊑ Doctor   Clinician ⊑ Staff
Car ⊑ Vehicle        SUV ⊑ Car              ElectricCar ⊑ Car       PetrolCar ⊑ Car
Van ⊑ Vehicle        WheelchairVan ⊑ Van    Ambulance ⊑ Van         Motorcycle ⊑ Vehicle
LowBatteryEV ⊑ ElectricCar                  AmbulanceBay ⊑ Bay      AccessibleBay ⊑ Bay
ChargerBay ⊑ Bay     LoadingBay ⊑ Bay       DropOffBay ⊑ Bay        StandardBay ⊑ Bay
FamilyBay ⊑ Bay      ClosedBay ⊑ Bay        GroundZone ⊑ Zone       UpperZone ⊑ Zone
```

The reasoner infers 15 further subsumptions by transitivity, for example OnCallDoctor ⊑ Person, LowBatteryEV ⊑ Vehicle and WheelchairVan ⊑ Vehicle.

### 4.2.4 Disjointness axioms

```
Staff ⊓ Visitor ⊑ ⊥        Staff ⊓ Supplier ⊑ ⊥        Patient ⊓ Supplier ⊑ ⊥     Visitor ⊓ Supplier ⊑ ⊥
Doctor ⊓ Nurse ⊑ ⊥         Doctor ⊓ Paramedic ⊑ ⊥      Nurse ⊓ Paramedic ⊑ ⊥      Paramedic ⊓
AdminOfficer ⊑ ⊥
Car ⊓ Van ⊑ ⊥              Car ⊓ Motorcycle ⊑ ⊥        Van ⊓ Motorcycle ⊑ ⊥       ElectricCar ⊓
PetrolCar ⊑ ⊥
SUV ⊓ ElectricCar ⊑ ⊥      SUV ⊓ PetrolCar ⊑ ⊥         AmbulanceBay ⊓ AccessibleBay ⊑ ⊥
AmbulanceBay ⊓ StandardBay ⊑ ⊥   GroundZone ⊓ UpperZone ⊑ ⊥   ClearedPerson ⊓ BannedPerson ⊑ ⊥
Person ⊓ Vehicle ⊑ ⊥       Vehicle ⊓ Bay ⊑ ⊥           ... (27 in all)
```

### 4.2.5 Definitions and restrictions

```
Clinician        ≡ Doctor ⊔ Nurse
OnCallDoctor     ≡ Doctor ⊓ ∃onCall.{true}
EmergencyVehicle ≡ Ambulance ⊓ ∃onRun.{true}
LowBatteryEV     ≡ ElectricCar ⊓ ∃battery.(< 30)
TallVehicle      ≡ Vehicle ⊓ ∃height.(> 210)
PetrolCar        ≡ Car ⊓ ¬ElectricCar ⊓ ¬SUV
UpperZone        ≡ Zone ⊓ ∃onLevel.(∃clearance.(≤ 210))
ClearedPerson    ≡ (Staff ⊓ ∃badge.ValidBadge ⊓ ¬BannedPerson) ⊔ ((Patient ⊔ Visitor) ⊓ ¬BannedPerson)
                   ⊔ (Supplier ⊓ ∃slot.(∃ arrivalTime))
AdmittedVehicle  ≡ EmergencyVehicle ⊔ ∃declared.Emergency ⊔ (Identified ⊓ ∃drivenBy.ClearedPerson)
AmbulanceBay  ⊑ ∀occupiedBy.EmergencyVehicle        ChargerBay ⊑ ∀occupiedBy.ElectricCar
AccessibleBay ⊑ ∀occupiedBy.(∃drivenBy.BadgeHolder) TallVehicle ⊑ ∀parkedIn.(∃inZone.GroundZone)
Vehicle ⊑ (≤ 1 drivenBy.Person)   Bay ⊑ (≤ 1 occupiedBy.Vehicle)   Vehicle ⊑ (≤ 1 parkedIn.Bay)
```

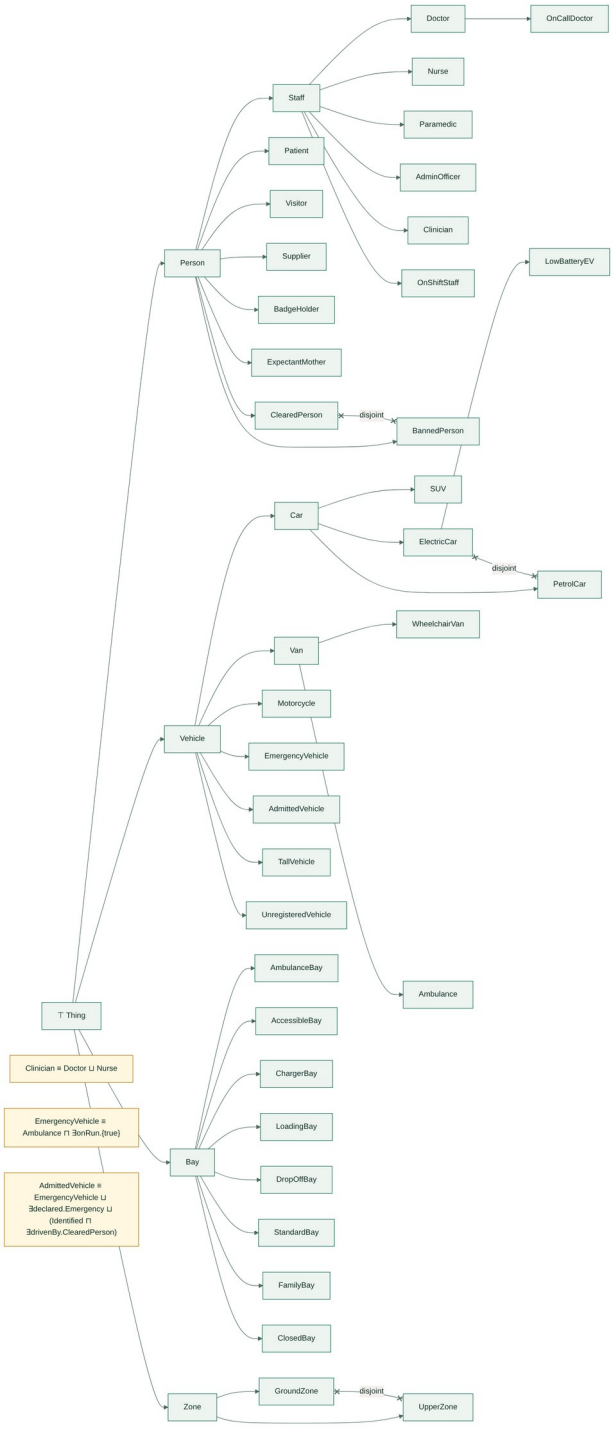



**Figure 5: DL concept hierarchy with disjointness and definitions**

### 4.2.6 Membership assertions (ABox) and realisation

Assertions come from the facts: `doctor(saad)` gives Doctor(saad), `bay_type(amb1, amb)` gives AmbulanceBay(amb1), `drives(noor, wv_no)` gives drives(noor, wv_no), and a height over 210 cm gives TallVehicle. Realisation finds the most specific concepts of each individual.

**Table 22: Realisation of selected individuals**

| Individual | Most specific concepts |
|---|---|
| `saad` | OnCallDoctor, OnShiftStaff, ClearedPerson |
| `noor` | Patient, BadgeHolder, ClearedPerson |
| `rukhsana` | Visitor, ExpectantMother, ClearedPerson |
| `waqar` | Visitor, BannedPerson |

| Individual | Most specific concepts |
|---|---|
| bushra | AdminOfficer, OnShiftStaff (not ClearedPerson: badge expired) |
| ev_sa | LowBatteryEV, AdmittedVehicle |
| wv_no | WheelchairVan, TallVehicle, AdmittedVehicle |
| amb_fa | Ambulance, EmergencyVehicle, TallVehicle, AdmittedVehicle |
| car_ya | PetrolCar (not admitted: plate unread) |
| amb1, ev2, z_patient | AmbulanceBay, ChargerBay, UpperZone |

### 4.2.7 DL consistency and interpretation

The consistency check looks for an individual that belongs to two disjoint concepts and for a ∀-restriction broken by an occupant (for example a non-emergency car in an ambulance bay). On the baseline day the result is "ABox consistent". In scenario tc20 Sadia's car is recorded as both a car and an SUV; it is classified as PetrolCar and SUV at once, so the reasoner reports `clash(car_sd, SUV, PetrolCar)` and the record is rejected (IC6 agrees).

Under the standard interpretation each concept is a set of individuals of the day and each role a set of pairs. For example AdmittedVehicle is interpreted as {ev_sa, mc_ma, suv_ar, car_im, van_ju, amb_fa, wv_no, car_ks, car_sd, car_ru, car_gh, car_th}: the 12 vehicles the rules admit. The DL view and the rule view agree, which is tested.

# 5. HORN CLAUSE KNOWLEDGE BASE

## 5.1 Knowledge base

The executable knowledge base is a set of definite Horn clauses with negation as failure in some bodies. Rules are stored as data, `r(Id, Head, Body)` in kb/rules.pl, so the forward chainer, the backward chainer, the exporter and the website all read the same 97 rules. A body literal is a fact pattern, `not(G)`, or a builtin (lt, le, gt, ge, eq, neq, add, sub, mul, cdiv) that is evaluated once its inputs are bound.

## 5.2 Facts

The 431 ground facts use 42 predicates.

**Table 23: Facts by predicate**

| Predicate | Count | Predicate | Count | Predicate | Count |
|---|---|---|---|---|---|
| walk/3 | 104 | drives/2 | 18 | shift/3 | 8 |
| bay_type/2 | 29 | vclass/2 | 18 | staff_badge/3 | 8 |
| bay_zone/2 | 29 | height/2 | 18 | visitor/1 | 6 |
| class_fits/2 | 21 | stay/2 | 18 | level_floor, level_route, route_clearance | 4 each |
| anpr/2 | 15 | registered/1 | 18 | doctor, patient, appointment, occupies, since | 3 each |
| arrival/2 | 15 | destination/2 | 18 | admin, nurse, battery, visiting_window | 2 each |
| zone_kind/2, zone_level/2 | 13 each | kind_penalty/2 | 13 | 15 others (today, mode, weather, on_call, security_ban …) | 1 each |

```
driver(saad, doctor, cardiology, ev_sa, ev, 150, 98, 425, 600).   % expands to role, drives, vclass,
on_call(saad).   battery(ev_sa, 22).   staff_badge(saad, 20250101, 20271231).  % height, anpr, arrival,
stay
shift(saad, 420, 1140).   appointment(noor, oncology, 600).   delivery_slot(junaid, 510, 570).
security_ban(waqar).   emergency_run(amb_fa).   emergency_declared(car_gh).   disability_badge(noor).
expectant(rukhsana).   bay(amb1, z_amb, amb).   zone(z_amb, g, ambulance).   level(l1, 1, 210).
walk(z_mat, maternity, 35).   visiting_window(660, 780).   out_of_service(vs4).   today(20261110).
```

## 5.3 Rules

**Table 24: Rule groups**

| Group | Rules | Derives | Phase |
|---|---|---|---|
| h | h1–h18 | taxonomy: staff_member, clinician, person, vehicle, ev, car_like, emergency_vehicle, unregistered (NAF) | 1 |
| s | s1–s7 | sensing: identified, battery_low, charger_bay, badge_valid, banned, code_red, raining | 1 |
| t | t1–t4 | time windows: on_shift, in_appointment_window, visiting_time, delivery_window | 1 |
| a | a1–a7 | cleared(P), admitted(V) | 1 |
| f | f1–f3 | fits(V,B), too_tall(V,L), bay_usable(B) | 1 |
| e | e1–e15 | entitled(V,Z): one or more rules per zone kind, code red variants | 1 |
| c | c1 | candidate(V,B) | 1 |
| p | p1–p10 | priority(V,0..5) | 1 |
| k | k1–k11 | adjusted, ev_adjust, zone_cost, cost(V,B,metres) | 1 |
| m | m1–m7 | fee(V, PKR) | 1 |
| w | w1–w14 | monitoring: occupant, ambulance_bay_blocked, tow_request, charger_hog, dropoff_limit, dropoff_overstay, manual_check, security_alert, notify_ward, route, alert_trauma_team, height_warning | 2 |

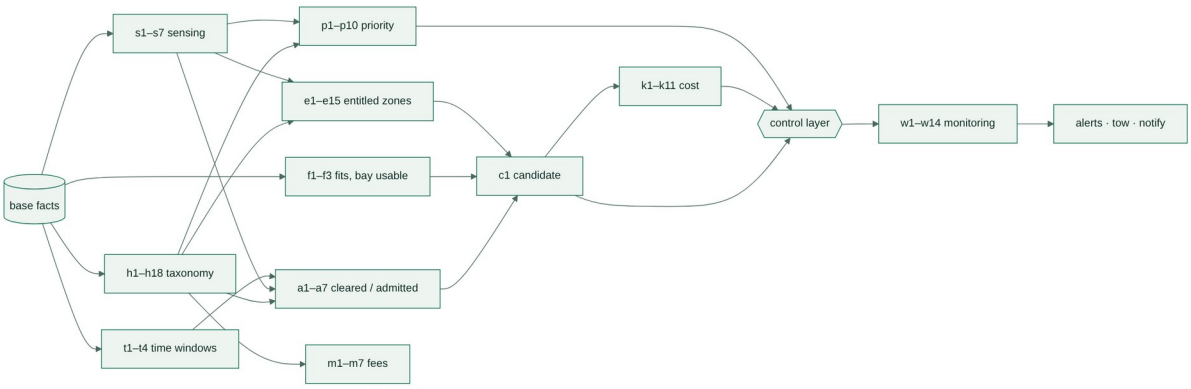



**Figure 6: Dependencies between rule groups**

Integrity constraints are goals that must have no solution. When one has, the checker prints the witness.

**Table 25: Integrity constraints**

| Id | Violation | Goal |
|---|---|---|
| IC1 | one bay, two vehicles at the same time | `alloc(V1,B,F1,T1), alloc(V2,B,F2,T2), V1 \= V2, F1 < T2, F2 < T1` |
| IC2 | one vehicle, two bays | `alloc(V,B1,_,_), alloc(V,B2,_,_), B1 \= B2` |
| IC3 | vehicle taller than its route clearance | `alloc(V,B,_,_), height(V,H), bay_zone(B,Z), zone_level(Z,L), route_clearance(L,C), H > C` |
| IC4 | vehicle class cannot use the bay type | `alloc(V,B,_,_), vclass(V,C), bay_type(B,T), not class_fits(C,T)` |
| IC5 | vehicle driven by two people | `drives(P1,V), drives(P2,V), P1 \= P2` |
| IC6 | vehicle with two classes | `vclass(V,C1), vclass(V,C2), C1 \= C2` |
| IC7 | bay in two zones | `bay_zone(B,Z1), bay_zone(B,Z2), Z1 \= Z2` |
| IC8 | shift ends before it starts | `shift(P,S,E), E =< S` |
| IC9 | badge expires before it is issued | `staff_badge(P,I,E), E < I` |
| IC10 | appointment for someone who is not a patient | `appointment(P,_,_), not patient(P)` |
| IC11 | ambulance not driven by a paramedic | `vclass(V,ambulance), drives(P,V), not paramedic(P)` |
| IC12 | vehicle in a bay that is out of service | `occupies(V,B), out_of_service(B)` |
| IC13 | staff role and visitor role on one person | `staff_member(P), visitor(P)` |

## 5.4 Horn clause conversion

A Horn clause is a disjunction of literals with at most one positive literal, written `Head :- Body`. Each FOL rule is converted in four steps: (1) remove → and ↔ (keeping the ← direction of a definition); (2) push ¬ inward; (3) drop the universal quantifiers, since clause variables are implicitly universal, and turn existentials in the body into body-only variables; (4) split a disjunction in the body, or a conjunction in the head, into separate clauses.

**Table 26: Conversion of FOL rules to Horn clauses**

| FOL statement | Clausal form | Horn clause(s) |
|---|---|---|
| f1: ∀x (Doctor(x) ∨ Nurse(x) ∨ Admin(x) ∨ Paramedic(x) → Staff(x)) | (¬Doctor(x) ∨ Staff(x)) ∧ (¬Nurse(x) ∨ Staff(x)) ∧ (¬Admin(x) ∨ Staff(x)) ∧ (¬Paramedic(x) ∨ Staff(x)) | h1 staff_member(P) :- doctor(P). … h4 staff_member(P) :- paramedic(P). |
| f6: ∀v (∃c (Anpr(v,c) ∧ c ≥ 90) → Identified(v)) | ¬Anpr(v,c) ∨ ¬(c ≥ 90) ∨ Identified(v) | s1 identified(V) :- anpr(V,C), ge(C,90). |
| f12: ∀p (Staff(p) ∧ BadgeValid(p) ∧ ¬Banned(p) → Cleared(p)) | ¬Staff(p) ∨ ¬BadgeValid(p) ∨ Banned(p) ∨ Cleared(p): two positive literals | a1 cleared(P) :- staff_member(P), badge_valid(P), not banned(P). (¬Banned as NAF) |

| FOL statement | Clausal form | Horn clause(s) |
|---|---|---|
| f13: ∀v (EmergencyVehicle(v) ∨ DeclaredEmergency(v) → Admitted(v)) | (¬EmergencyVehicle(v) ∨ Admitted(v)) ∧ (¬DeclaredEmergency(v) ∨ Admitted(v)) | a5 admitted(V) :- emergency_vehicle(V).<br>a6 admitted(V) :- emergency_declared(V), arrival(V,_). |
| f14: ∀v ∀p (Arrival(v) ∧ Identified(v) ∧ Drives(p,v) ∧ Cleared(p) → Admitted(v)) | ¬Arrival(v) ∨ ¬Identified(v) ∨ ¬Drives(p,v) ∨ ¬Cleared(p) ∨ Admitted(v) | a7 admitted(V) :- arrival(V,_), identified(V), drives(P,V), cleared(P). |
| f20: Candidate(v,b) ↔ Admitted(v) ∧ Fits(v,b) ∧ ∃z (InZone(b,z) ∧ Entitled(v,z)) ∧ Usable(b) | ← direction: one clause; → direction by closed-world completion | c1 candidate(V,B) :- admitted(V), fits(V,B), bay_zone(B,Z), entitled(V,Z), bay_usable(B). |
| f3: ∀x (Person(x) → Staff(x) ∨ Patient(x) ∨ Visitor(x) ∨ Supplier(x)) | ¬Person(x) ∨ Staff(x) ∨ Patient(x) ∨ Visitor(x) ∨ Supplier(x): four positive literals | not Horn: kept as a model check |
| f35: ∀x ¬(Staff(x) ∧ Visitor(x)) | ¬Staff(x) ∨ ¬Visitor(x): no positive literal (goal clause) | IC13 :- staff_member(P), visitor(P). |

## 5.5 Prolog form

The key rules as they appear in kb/rules.pl (printed by `swipl main.pl rules`):

```
s1  identified(V)  :- anpr(V,C), ge(C,90).
s4  badge_valid(P) :- staff_badge(P,I,E), today(T), le(I,T), ge(E,T).
t1  on_shift(P)    :- shift(P,S,E), drives(P,V), arrival(V,T), sub(S,30,S0), ge(T,S0), le(T,E).
t2  in_appointment_window(P) :- appointment(P,_,T0), drives(P,V), arrival(V,T),
                                sub(T0,60,A), add(T0,30,B), ge(T,A), le(T,B).
t3  visiting_time(V) :- arrival(V,T), visiting_window(A,B), ge(T,A), lt(T,B).
a1  cleared(P)  :- staff_member(P), badge_valid(P), not banned(P).
a3  cleared(P)  :- visitor(P), not banned(P).
a5  admitted(V) :- emergency_vehicle(V).
a7  admitted(V) :- arrival(V,_), identified(V), drives(P,V), cleared(P).
f1  fits(V,B)   :- vclass(V,C), bay_type(B,T), class_fits(C,T), height(V,H),
                   bay_zone(B,Z), zone_level(Z,L), route_clearance(L,Cl), le(H,Cl).
e3  entitled(V,Z) :- drives(P,V), disability_badge(P), admitted(V), zone_kind(Z,accessible).
e11 entitled(V,Z) :- drives(P,V), visitor(P), visiting_time(V), admitted(V),
                     not code_red, zone_kind(Z,visitor).
c1  candidate(V,B) :- admitted(V), fits(V,B), bay_zone(B,Z), entitled(V,Z), bay_usable(B).
p1  priority(V,0)  :- emergency_vehicle(V).
p3  priority(V,1)  :- drives(P,V), on_call(P), on_shift(P), admitted(V).
k3  ev_adjust(V,B,120) :- candidate(V,B), battery_low(V), not charger_bay(B).
k8  zone_cost(overflow,P) :- kind_penalty(overflow,P0), raining, add(P0,300,P).
k9  cost(V,B,C) :- candidate(V,B), bay_zone(B,Z), drives(P,V), destination(P,D),
                   walk(Z,D,W), zone_kind(Z,K), zone_cost(K,Pen), ev_adjust(V,B,A),
                   add(W,Pen,C1), add(C1,A,C).
w3  ambulance_bay_blocked(B) :- occupant(V,B), bay_type(B,amb), not emergency_vehicle(V).
w4  tow_request(B) :- ambulance_bay_blocked(B), occupant(V,B), unregistered(V).
w8  dropoff_overstay(V) :- assigned(V,B), bay_type(B,drop), stay(V,M),
                           dropoff_limit(V,L), gt(M,L).
```

# 6. INFERENCE AND REASONING

The same rules are run in two directions. The forward chainer starts from the facts and derives everything that follows, which is how the gate decides a whole day. The backward chainer starts from one question ("is car_bu admitted?") and works back to the facts, which is how a single decision is checked and explained.

## 6.1 Forward Chaining

### 6.1.1 Algorithm

1. **Stratify.** A predicate used under `not` must be complete before it is negated. Relaxation over the rule dependency graph places each predicate in a stratum: 3 strata (0, 1, 2) here. For example banned is in stratum 0 and cleared, which tests not banned, in stratum 1.
2. **Saturate each stratum in turn.** In each cycle every rule of the stratum is matched against a snapshot of working memory; builtins are evaluated once bound and not(G) succeeds only if G is not in memory. Each new head instance is added and logged as fire(Cycle, Stratum, Rule, Fact, Bindings). The stratum ends when a cycle adds nothing.
3. **Replay the day (control layer).** Events are taken in time order. At each minute departures go first, then arrivals by priority (0 first) and arrival time. Each vehicle gets its cheapest free candidate bay, ties broken by bay id; a vehicle with no free bay waits and is retried after every departure. This produces assigned(V,B).
4. **Phase 2.** The monitoring rules (w1–w14) saturate over the facts plus assigned/2, in their own strata.
5. **Check** the 13 integrity constraints and the FOL model checks.

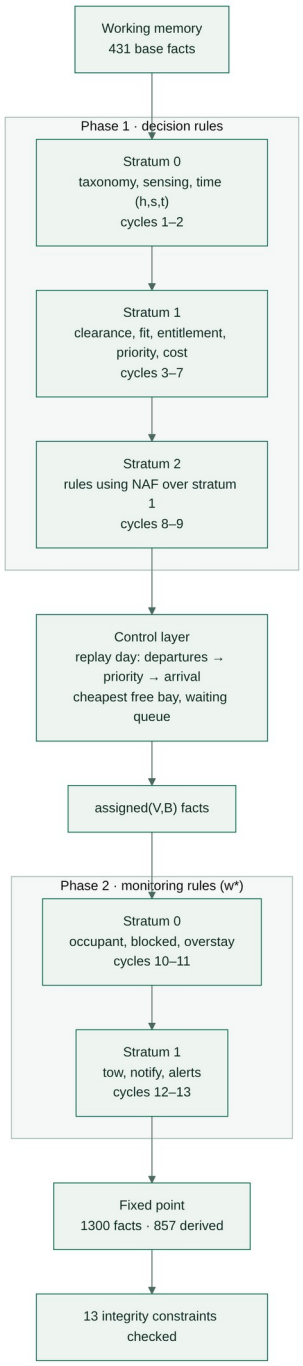



**Figure 7: Forward chaining: two phases of stratified saturation with the control layer between them**

### 6.1.2 Initial facts

Working memory starts with the 431 facts of section 5.2. For the trace that follows, the relevant ones are: `visitor(rukhsana)`, `expectant(rukhsana)`, `drives(rukhsana, car_ru)`, `vclass(car_ru, suv)`, `height(car_ru, 180)`, `anpr(car_ru, 95)`, `arrival(car_ru, 700)`, `stay(car_ru, 240)`, `destination(rukhsana, maternity)`, `visiting_window(660, 780)`, the bay, zone and level facts, and `walk(z_mat, maternity, 35)`.

### 6.1.3 Inference cycles

**Table 27: Cycles and strata on the baseline day**

| Phase · stratum | Cycles | New facts per cycle | What fires |
|---|---|---|---|
| 1 · s0 | 1–2 | 427, 11 | taxonomy, sensing, time windows, fits |
| 1 · s1 | 3–7 | 62, 24, 70, 58, 11 | cleared → admitted → entitled, priority, fee → candidate |

| Phase · stratum | Cycles | New facts per cycle | What fires |
|---|---|---|---|
| 1 · s2 | 8–9 | 82, 63 | ev_adjust (uses not charger_bay) → cost |
| 2 · s0 | 10–11 | 33, 1 | occupant, route, height warnings, alerts |
| 2 · s1 | 12–13 | 13, 2 | dropoff_limit → dropoff_overstay, manual_check |
| Total | 13 | 857 | 431 → 1,300 facts at the fixed point |

### 6.1.4 Rules activated and new facts

The trace below follows one decision, how Rukhsana gets bay mt1, through every cycle (from docs/generated/01_forward_trace.txt):

```
[c1 s0] h9   person(rukhsana)                [P=rukhsana]
[c1 s0] s1   identified(car_ru)              [V=car_ru, C=95]            95 >= 90
[c1 s0] t3   visiting_time(car_ru)           [T=700, A=660, B=780]       11:40 in 11:00-13:00
[c1 s0] f1   fits(car_ru,mt1)                [C=suv, T=family, H=180, L=l1, Cl=210]
[c3 s1] a3   cleared(rukhsana)               visitor, not banned
[c4 s1] a7   admitted(car_ru)                [V=car_ru, P=rukhsana]
[c5 s1] e2   entitled(car_ru,z_drop)
[c5 s1] e10  entitled(car_ru,z_mat)          expectant mother
[c5 s1] e11  entitled(car_ru,z_visit)        visiting time, not code red
[c5 s1] e15  entitled(car_ru,z_roof)
[c5 s1] p5   priority(car_ru,2)              (p9 gives 4 and p10 gives 5; lowest wins)
[c5 s1] m7   fee(car_ru,250)                 [M=240 min: 100 + 3 x 50]
[c6 s1] c1   candidate(car_ru,mt1)           ... and dr1, dr2, vs1-vs3, rf1-rf3
[c8 s2] k5   ev_adjust(car_ru,mt1,0)         not an EV
[c9 s2] k9   cost(car_ru,mt1,35)             [W=35, Pen=0, A=0]
[c9 s2] k9   cost(car_ru,vs1,80)             [W=80]
[c9 s2] k9   cost(car_ru,rf1,210)            [W=150, Pen=60]
[c9 s2] k9   cost(car_ru,dr1,480)            [W=180, Pen=300]
control 11:40   car_ru -> mt1   (cheapest free candidate, cost 35, priority 2)
[c10 s0] w11 notify_ward(car_ru,maternity)
```

### 6.1.5 Final conclusion

**Table 28: Decisions on the baseline day**

| Time | Vehicle | Decision | Cost (m) | Priority |
|---|---|---|---|---|
| 06:40 | mc_ma | mc1 | 170 | 3 |
| 07:05 | ev_sa | ev1 | 150 | 1 |
| 07:45 | car_im | pt1 | 40 | 2 |
| 07:50 | suv_ar | st3 | 70 | 3 |
| 08:40 | van_ju | ld1 | 30 | 3 |
| 09:00 | amb_fa | amb1 | 10 | 0 |
| 09:20 | car_bu | denied: badge_invalid | — | — |
| 09:45 | wv_no | ac1 | 140 | 2 |
| 10:15 | car_ks | dr1 | 460 | 4 |
| 11:30 | car_sd | fm1 | 85 | 4 |
| 11:40 | car_ru | mt1 | 35 | 2 |
| 11:45 | car_wq | denied: security_ban | — | — |
| 12:40 | car_gh | dr1 | 20 | 0 |
| 14:00 | car_th | dr1 | 420 | 5 |
| 15:05 | car_ya | denied: plate_unreadable | — | — |

Phase 2 raises: charger_hog(ev_zb, ev2), dropoff_overstay(car_ks), dropoff_overstay(car_th), manual_check(car_ya), security_alert(waqar), notify_ward(car_ru, maternity), alert_trauma_team(car_gh), and height warnings for van_ju, amb_fa and wv_no. Fees: car_ru PKR 250, car_sd 150, car_im 100, car_ks 100, wv_no 50; staff on shift, emergencies and the supplier pay nothing. No integrity constraint is violated.

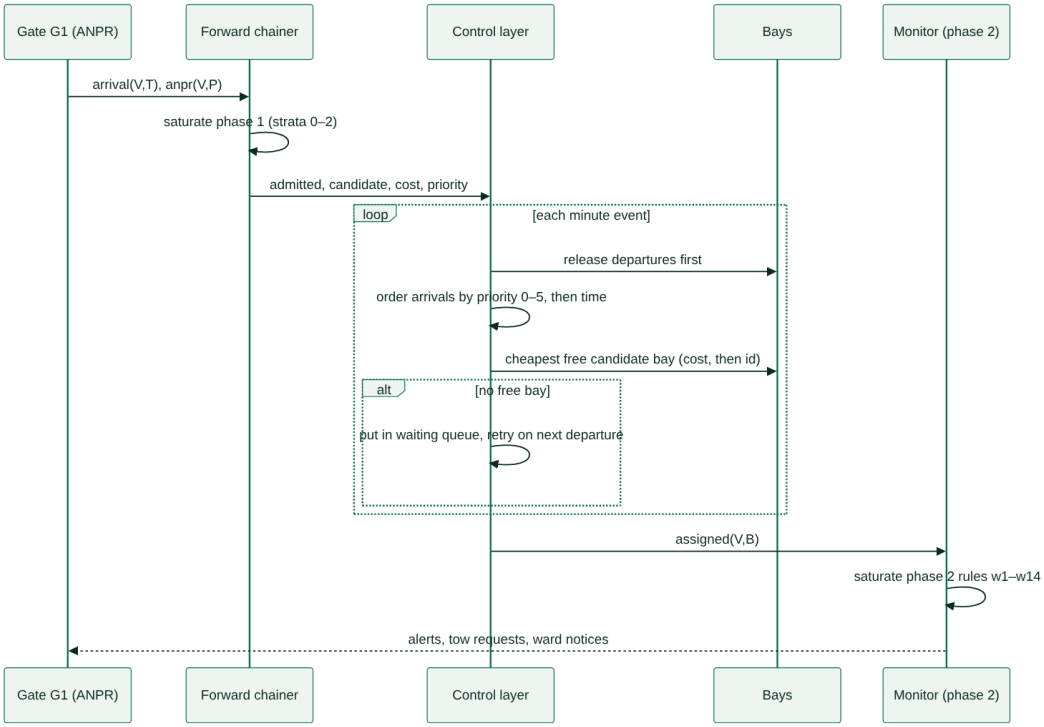



**Figure 8: Day replay: gate, forward chainer, control layer, bays and monitor**

### 6.1.6 Selected firings by cycle

**Table 29: Selected rule firings (full list: docs/generated/01_forward_trace.txt, 857 firings)**

| Cycle | Stratum | Rule | New fact |
|---|---|---|---|
| 1 | 0 | h1 | `staff_member(saad)` |
| 1 | 0 | h17 | `emergency_vehicle(amb_fa)` |
| 1 | 0 | s1 | `identified(ev_sa)` |
| 1 | 0 | s2 | `battery_low(ev_sa)` |
| 1 | 0 | s4 | `badge_valid(saad)` |
| 1 | 0 | s5 | `banned(waqar)` |
| 1 | 0 | t1 | `on_shift(saad)` |
| 1 | 0 | t2 | `in_appointment_window(imtiaz)` |
| 1 | 0 | t4 | `delivery_window(junaid)` |
| 2 | 0 | h7 | `person(saad)` |
| 3 | 1 | a1 | `cleared(saad)` |
| 3 | 1 | a2 | `cleared(noor)` |
| 3 | 1 | h18 | `unregistered (none on the baseline day)` |
| 4 | 1 | a5 | `admitted(amb_fa)` |
| 4 | 1 | a6 | `admitted(car_gh)` |
| 4 | 1 | a7 | `admitted(wv_no)` |
| 5 | 1 | e1 | `entitled(amb_fa, z_amb)` |
| 5 | 1 | e3 | `entitled(wv_no, z_acc)` |
| 5 | 1 | e5 | `entitled(ev_sa, z_ev)` |
| 5 | 1 | e8 | `entitled(ev_sa, z_oncall)` |

| Cycle | Stratum | Rule | New fact |
|---|---|---|---|
| 5 | 1 | p1 | priority(amb_fa, 0) |
| 5 | 1 | p3 | priority(ev_sa, 1) |
| 6 | 1 | c1 | candidate(ev_sa, ev1) |
| 8 | 2 | k3 | ev_adjust(ev_sa, oc1, 120) |
| 9 | 2 | k9 | cost(ev_sa, ev1, 150) |
| 9 | 2 | k11 | cost(car_gh, dr1, 20) |
| 10 | 0 | w5 | charger_hog(ev_zb, ev2) |
| 10 | 0 | w13 | alert_trauma_team(car_gh) |
| 10 | 0 | w14 | height_warning(wv_no) |
| 13 | 1 | w8 | dropoff_overstay(car_ks) |
| 13 | 1 | w9 | manual_check(car_ya) |

## 6.2 Backward Chaining

### 6.2.1 Algorithm

The backward chainer (engine/backward.pl) is a meta-interpreter over the same rule data. It records a Prolog box-model trace with the ports CALL, EXIT, REDO and FAIL, plus TRY for each rule, and returns a proof tree.

1. solve(G): if the depth exceeds 40, fail. If G is not(A), try to prove A: success means not(A) fails, failure means it succeeds (negation as failure).
2. If G is a builtin, evaluate it once its inputs are bound: EXIT or FAIL.
3. Otherwise log CALL. Try the stored facts first, then each rule r(Id,H,B) in source order: rename its variables, unify H with G, log TRY, and solve the body left to right.
4. On success log EXIT; when the caller backtracks log REDO and try the next fact or rule; when nothing is left log FAIL.
5. For monitoring goals (w1–w14) the store also holds the assigned(V,B) facts produced by the day replay.
6. why_not(G): for a failed goal, take every rule whose head matches, find the first body literal that cannot be proved, and explain that literal the same way, down to a missing fact or a false comparison.

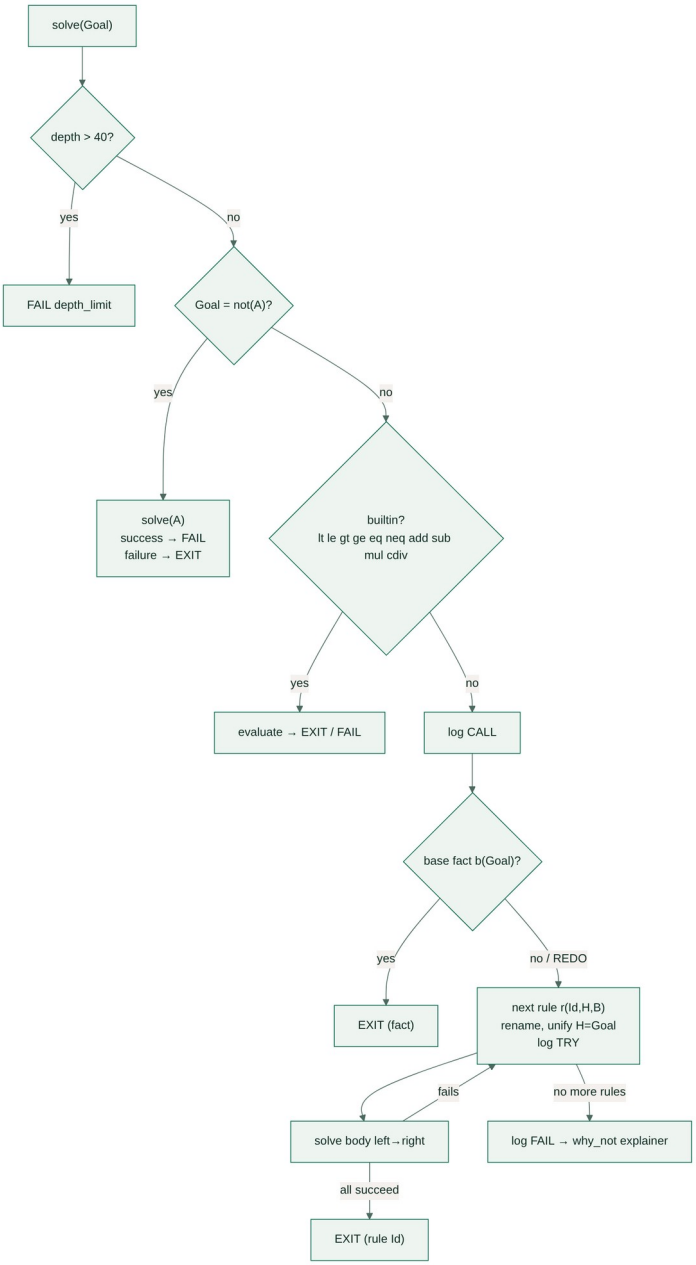



**Figure 9: Backward chaining meta-interpreter**

### 6.2.2 Successful proof

Goal: `admitted(wv_no)`, is Noor's wheelchair van admitted? (docs/generated/03_backward_admitted_wv_no.txt)

```
CALL  admitted(wv_no)
      try a5: emergency_vehicle(wv_no)
|  CALL  emergency_vehicle(wv_no)
|        try h17: vclass(wv_no,ambulance), emergency_run(wv_no)
|  |  CALL  vclass(wv_no,ambulance)
|  |  FAIL  vclass(wv_no,ambulance)
|  FAIL  emergency_vehicle(wv_no)
      try a6: emergency_declared(wv_no), arrival(wv_no,_)
|  CALL  emergency_declared(wv_no)
|  FAIL  emergency_declared(wv_no)
      try a7: arrival(wv_no,_), identified(wv_no), drives(_P,wv_no), cleared(_P)
|  CALL  arrival(wv_no,_)           EXIT  arrival(wv_no,585)
|  CALL  identified(wv_no)
```

```
|       try s1: anpr(wv_no,_C), ge(_C,90)
|  |  CALL  anpr(wv_no,_C)         EXIT  anpr(wv_no,94)
|  |  CALL  ge(94,90)              EXIT  ge(94,90)
|  EXIT  identified(wv_no)
|  CALL  drives(_P,wv_no)          EXIT  drives(noor,wv_no)
|  CALL  cleared(noor)
|       try a1: staff_member(noor), badge_valid(noor), not banned(noor)
|  |  CALL  staff_member(noor)     h1-h4 tried: doctor/nurse/admin/paramedic(noor) all FAIL
|  |  FAIL  staff_member(noor)
|       try a2: patient(noor), not banned(noor)
|  |  CALL  patient(noor)          EXIT  patient(noor)
|  |  CALL  not banned(noor)
|  |  |  CALL  banned(noor)        try s5: security_ban(noor)  FAIL
|  |  |  FAIL  banned(noor)
|  |  EXIT  not banned(noor)
|  EXIT  cleared(noor)
EXIT  admitted(wv_no)

Proof tree
admitted(wv_no)              <a7>
  arrival(wv_no,585)         <fact>
  identified(wv_no)          <s1>
    anpr(wv_no,94)           <fact>
    ge(94,90)                <builtin>
  drives(noor,wv_no)         <fact>
  cleared(noor)              <a2>
    patient(noor)            <fact>
    not banned(noor)         <naf>
```

A second successful proof uses the allocation: `dropoff_overstay(car_ks)` is proved by w8 from assigned(car_ks, dr1), bay_type(dr1, drop), stay(car_ks, 20), dropoff_limit(car_ks, 15) (by w7, since not emergency_declared(car_ks)) and the builtin gt(20, 15).

### 6.2.3 Failed proof with backtracking

Goal: `admitted(car_bu)`, is Bushra's car admitted? (docs/generated/04_backward_admitted_car_bu_fails.txt)

```
CALL  admitted(car_bu)
      try a5: emergency_vehicle(car_bu)
|  CALL  emergency_vehicle(car_bu)
|       try h17: vclass(car_bu,ambulance), emergency_run(car_bu)
|  |  CALL  vclass(car_bu,ambulance)    FAIL
|  FAIL  emergency_vehicle(car_bu)
      try a6: emergency_declared(car_bu), arrival(car_bu,_)
|  CALL  emergency_declared(car_bu)     FAIL
      try a7: arrival(car_bu,_), identified(car_bu), drives(_P,car_bu), cleared(_P)
|  CALL  arrival(car_bu,_)              EXIT  arrival(car_bu,560)
|  CALL  identified(car_bu)             s1: anpr(car_bu,97), ge(97,90)  EXIT
|  CALL  drives(_P,car_bu)              EXIT  drives(bushra,car_bu)
|  CALL  cleared(bushra)
|       try a1: staff_member(bushra), badge_valid(bushra), not banned(bushra)
|  |  CALL  staff_member(bushra)         h3: admin(bushra)  EXIT
|  |  CALL  badge_valid(bushra)
|  |       try s4: staff_badge(bushra,_I,_E), today(_T), le(_I,_T), ge(_E,_T)
|  |  |  CALL  staff_badge(bushra,_I,_E) EXIT  staff_badge(bushra,20241101,20261031)
|  |  |  CALL  today(_T)                 EXIT  today(20261110)
|  |  |  CALL  le(20241101,20261110)     EXIT
|  |  |  CALL  ge(20261031,20261110)     FAIL
```

```
|  |  |  REDO  today, staff_badge: no other facts
|  |  FAIL  badge_valid(bushra)
|  |  REDO  staff_member(bushra): no other rule
|        try a2: patient(bushra)          FAIL
|        try a3: visitor(bushra)          FAIL
|        try a4: supplier(bushra)         FAIL
|  FAIL  cleared(bushra)
|  REDO  drives, identified, arrival: no alternatives
FAIL  admitted(car_bu)

Why not: admitted(car_bu)
 admitted(car_bu) cannot be proved:
   a5 blocked at emergency_vehicle(car_bu)
     h17 blocked at vclass(car_bu,ambulance)   <- no such fact is stored
   a6 blocked at emergency_declared(car_bu)    <- no such fact is stored
   a7 blocked at cleared(bushra)
     a1 blocked at badge_valid(bushra)
       s4 blocked at ge(20261031,20261110)     <- comparison is false
     a2 blocked at patient(bushra)             <- no such fact is stored
     a3 blocked at visitor(bushra)             <- no such fact is stored
     a4 blocked at supplier(bushra)            <- no such fact is stored
```

**Table 30: Why-not explanations for the other refusals**

| Goal | First failing literal | Reason at the gate |
|---|---|---|
| `admitted(car_wq)` | a3 blocked at not banned(waqar): banned(waqar) holds, so its negation fails | security_ban |
| `admitted(car_ya)` | s1 blocked at ge(64,90): comparison is false | plate_unreadable, manual check |
| `fits(wv_no, pt1)` | f1 blocked at le(230,210): the 230 cm van exceeds the 210 cm ramp | used in scenario tc11 |

# 7. IMPLEMENTATION

## 7.1 Tools and Technologies

**Table 31: Tools and technologies**

| Tool | Use |
|---|---|
| SWI-Prolog 9 | Knowledge base, forward and backward chainers, frames, semantic network, FOL and DL modules, command-line program |
| plunit | 159 Prolog unit tests |
| JavaScript (ES2020) | Browser port of the reasoning engine (website/engine.js) |
| Node.js 18+ and node:test | 71 tests that the JavaScript engine matches Prolog on all 22 scenarios; site build |
| three.js 0.160 | 3D digital twin of the car park |
| HTML and CSS | Control-room web interface, light and dark themes, phone layout |
| Mermaid | The 10 diagrams in this report |

## 7.2 Code / Execution

### 7.2.1 Project structure

```
CarePark-KR/
  Read me first.txt          how to open and run everything
  README.md
  main.pl                    command-line program and menu
  kb/        facts.pl        431 facts: levels, zones, bays, walking table, drivers, badges, shifts
             rules.pl        97 Horn rules as data + 13 integrity constraints
             scenarios.pl    22 test scenarios as deltas with expected outcomes
  engine/    core.pl         rule loading, stratification, saturation with NAF and builtins
             allocate.pl     day replay: departures, priorities, cheapest bay, waiting queue, phase 2
             backward.pl     meta-interpreter, box-model trace, proof trees, why-not
             check.pl        integrity constraints and expected-outcome checks
             frames.pl       46 frames, facets, demons, PARKING-VISIT script
             semnet.pl       61 nodes, 79 links and graph queries
             fol.pl          36 FOL statements and model checks
             dl.pl           89 TBox axioms, ABox, realisation, consistency
             stats.pl        the project numbers
  tools/     export.pl       writes the knowledge base and results to website/*.json
             build_site.mjs  builds website/index.html
  tests/     run.pl + 4 suites (159 tests)       tests_js/  engine.test.mjs (71 tests)
  website/   index.html, engine.js, kb.json, results.json, site.json, src/ (page, app, 3D twin)
  docs/      diagrams/ (10), generated/ (14 trace sheets), screenshots/ (12)
  report/    this report (DOCX and PDF)
```

### 7.2.2 Knowledge base implementation

Facts are written compactly and expanded by `fact/1`. One `driver/9` line gives the role, destination, drives, vclass, height, anpr, arrival and stay facts; one `walk_row/2` line gives the eight walking distances of a zone. Rules are terms `r(Id, Head, Body)` read with their variable names, so traces print `V=car_ru` rather than internal variable numbers. Scenarios are deltas: `sc(Id, Title, Story, Remove, Add, ExpectedChanges, Checks)`.

### 7.2.3 Frame implementation

`get(Frame, Slot, Value, Source)` follows the lookup order of section 3.1.1 and also returns where the value came from (local value, if-needed demon, inherited, default). `put/3` stores a local value and runs any if-added demon. `isa_chain/2` computes the breadth-first inheritance order for multiple inheritance.

### 7.2.4 Forward chaining implementation

`strata/1` assigns each derived predicate a stratum by relaxation: a head is at least as high as each positive body predicate and strictly higher than each negated one. `saturate/3` runs stratum by stratum; within a stratum it takes a snapshot of working memory, matches every rule with `findall`, and adds the new facts, until a cycle adds nothing. `run_day/2` performs the control-layer replay and then saturates phase 2.

### 7.2.5 Backward chaining implementation

`solve/3` is the meta-interpreter of section 6.2.1. A global counter keeps the trace linear in its length, and a quiet mode switches tracing off for bulk checks. `why_not/2` builds the explanation tree; it stops at depth 12 to stay readable.

### 7.2.6 User interaction and sample queries

**Table 32: Commands**

| Command | Output |
|---|---|
| `swipl main.pl` | interactive menu |
| `swipl main.pl day [tc05]` | the day for the baseline or any scenario: timeline, decisions, alerts, fees, constraints |
| `swipl main.pl saturate` | full forward chaining trace |
| `swipl main.pl prove "admitted(wv_no)"` | backward trace and proof tree |
| `swipl main.pl whynot "admitted(car_bu)"` | why-not explanation |
| `swipl main.pl scenarios` | all 22 scenarios, pass or fail |
| `swipl main.pl frames \| net \| fol \| dl \| script \| rules \| facts \| stats` | each knowledge view |
| `swipl main.pl export` | regenerates the website data |
| `swipl tests/run.pl` | 159 Prolog tests |
| `node --test tests_js/engine.test.mjs` | 71 JavaScript tests |

### 7.2.7 Sample execution

```
$ swipl main.pl day
================================================================================
Baseline day
================================================================================
431 facts in, 808 derived in phase 1 (decisions), 49 in phase 2 (monitoring)

Decisions
  06:40  mc_ma        bay mc1, cost 170 m, priority 3, waited 0 min
  07:05  ev_sa        bay ev1, cost 150 m, priority 1, waited 0 min
  07:45  car_im       bay pt1, cost 40 m, priority 2, waited 0 min
  07:50  suv_ar       bay st3, cost 70 m, priority 3, waited 0 min
  08:40  van_ju       bay ld1, cost 30 m, priority 3, waited 0 min
  09:00  amb_fa       bay amb1, cost 10 m, priority 0, waited 0 min
  09:20  car_bu       DENIED (badge_invalid)
  ...
  15:05  car_ya       DENIED (plate_unreadable)

Alerts and guidance (phase 2)
  charger_hog(ev_zb,ev2)
  dropoff_overstay(car_ks)
  security_alert(waqar)
  notify_ward(car_ru,maternity)
  alert_trauma_team(car_gh)
  ...
Integrity constraints IC1-IC13: []
```

```
$ swipl main.pl scenarios
  tc01 Ordinary Tuesday (baseline)                                  pass []
  tc02 Code red at the emergency department                         pass []
  ...
22/22 scenarios pass

$ swipl tests/run.pl
% All 159 tests passed
$ node --test tests_js/engine.test.mjs
# pass 71
# fail 0
```

### 7.2.8 Execution screenshots

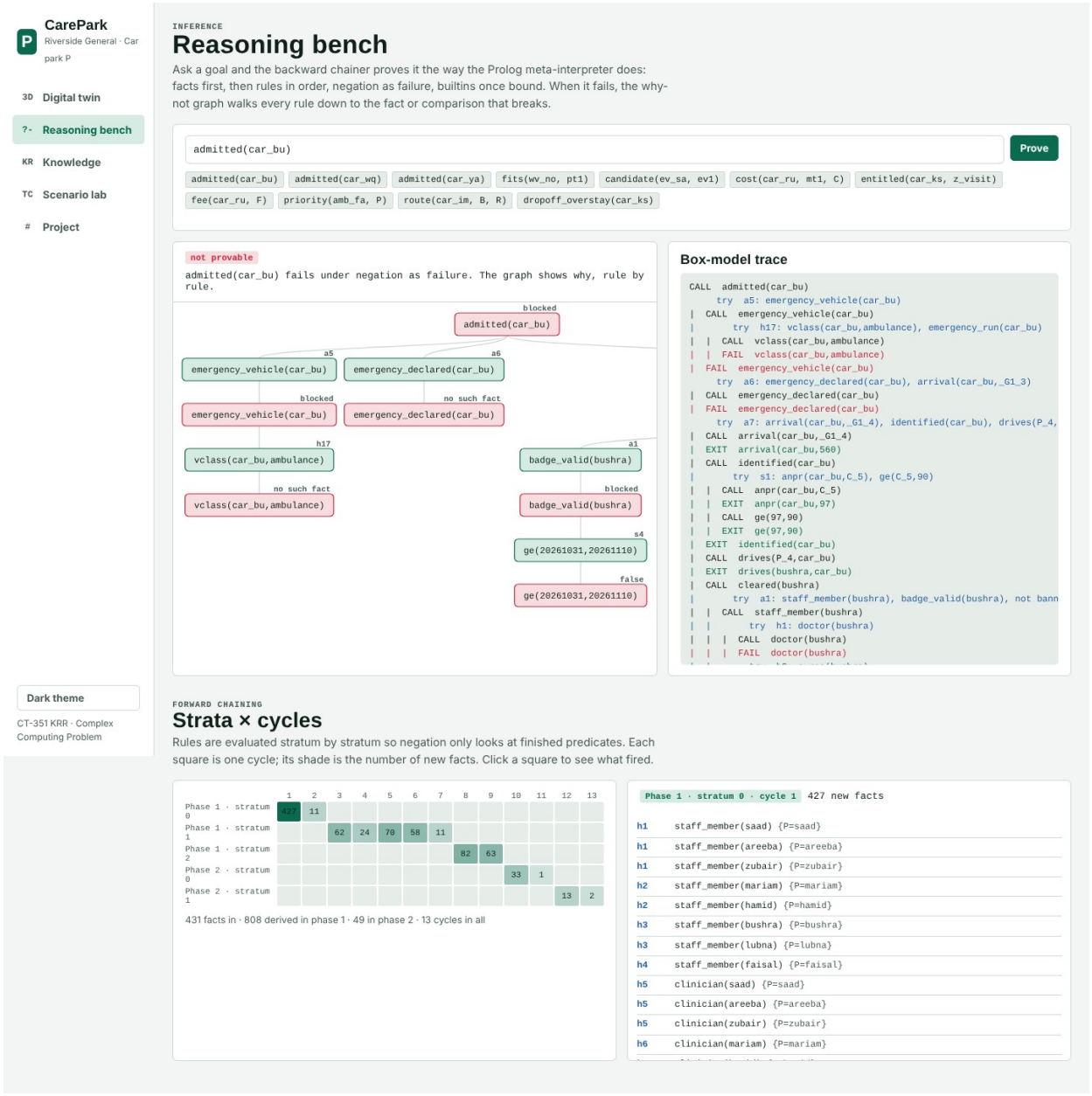



**Figure 10: Reasoning bench: why-not graph and box-model trace for admitted(car_bu), with the strata × cycles grid**

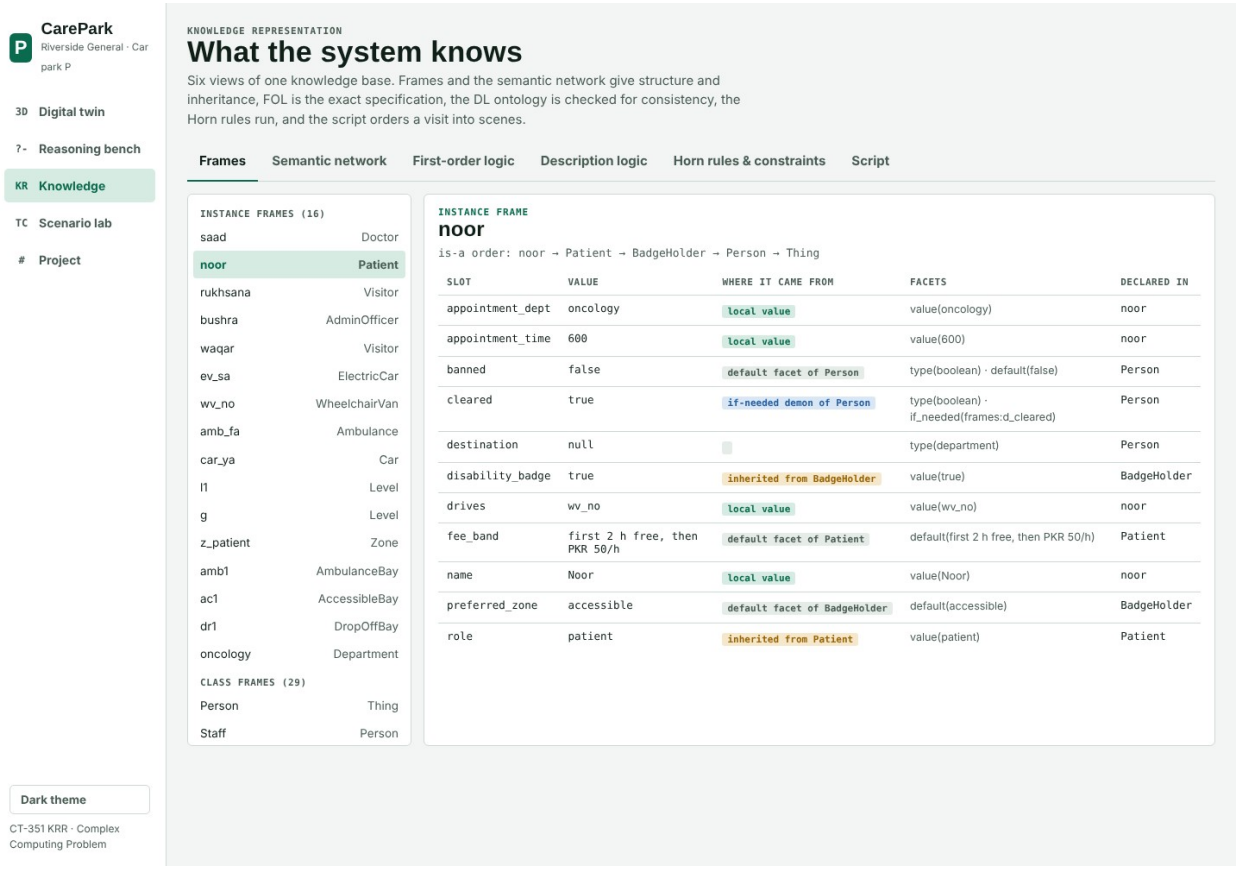



**Figure 11: Knowledge view: the noor frame with the source of every slot value**

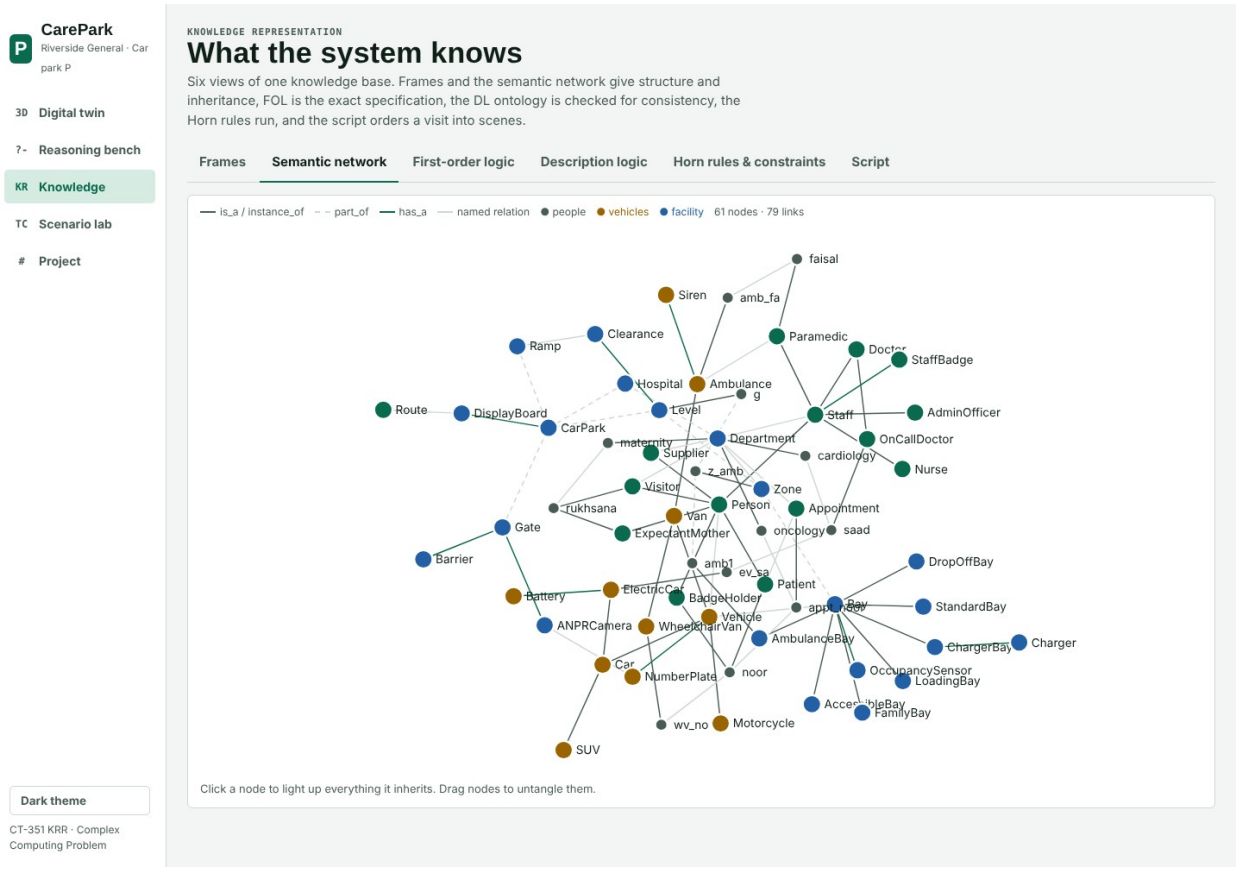



**Figure 12: Knowledge view: the semantic network**

### 7.2.9 Project website

The website, CarePark Control Room, opens on a 3D digital twin of Car park P: four levels, 29 bays, the gate with an ANPR barrier, the ramp and the hospital block. Pressing Play or dragging the timeline replays the day, and each car drives to its bay. Clicking a car shows its rule chain, every candidate bay with its cost in metres, the fee and the walking route to its department. The other views are the reasoning bench (any goal, proved in the browser), the knowledge views (frames, network, FOL, DL, rules, script), the scenario lab (22 cases with live pass or fail and a builder for new ones) and the project page. It runs the same rules as Prolog through engine.js, works in light and dark themes and on a phone.

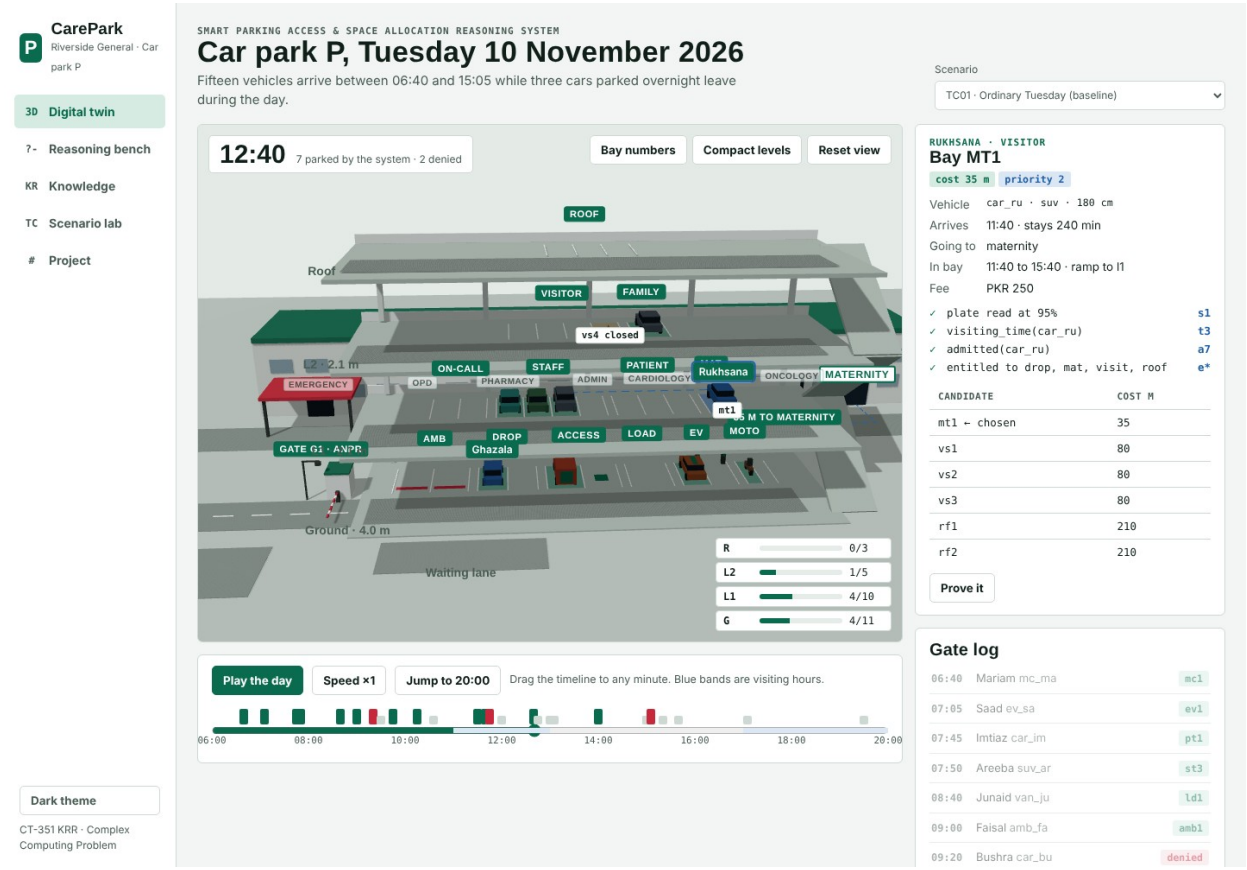



**Figure 13: Digital twin at 12:40 with Rukhsana selected in mt1, 35 m from maternity**

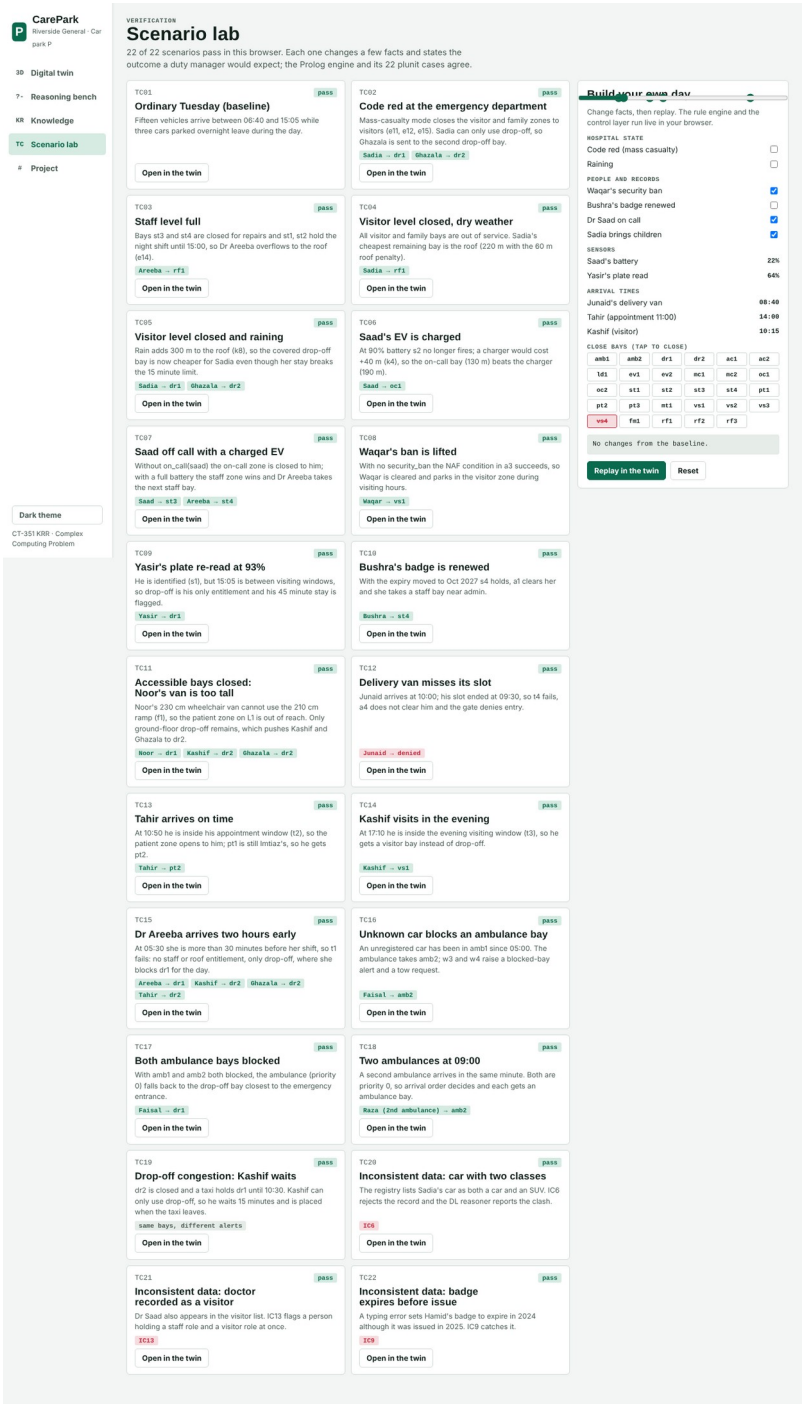



**Figure 14: Scenario lab: 22 cases checked live, and the scenario builder**

# 8. RESULTS AND DISCUSSION

## 8.1 Results

### 8.1.1 Decisions for the main scenario

On the baseline day 12 vehicles are parked and 3 refused, each for the reason the policy gives: Bushra's badge expired on 31 October (badge_invalid), Waqar is on the ban list (security_ban), and Yasir's plate was read at 64% (plate_unreadable, sent to a manual check). Every parked vehicle got its cheapest free candidate: the ambulance went to amb1 at 10 m from emergency, Saad's low-battery EV took a charger (ev1) even though an on-call bay is nearer, because the 120 m penalty for no charger made oc1 cost 170 m against 150 m, and Noor's 230 cm van was kept on the ground floor in the accessible bay ac1. Kashif arrived outside visiting hours and could only use drop-off, where his 20 minute stay was flagged.

### 8.1.2 Test cases

**Table 33: Test scenarios and results**

| Id | Scenario | What changes in the reasoning | Expected and obtained | Result |
|---|---|---|---|---|
| tc01 | Ordinary Tuesday (baseline) | — | 12 parked, 3 denied; 7 key alerts; no IC | Pass |
| tc02 | Code red at the emergency department | e11, e12, e15 close visitor zones to visitors | car_sd → dr1, car_gh → dr2; overstay(car_sd) | Pass |
| tc03 | Staff level full | st3, st4 closed; st1, st2 held; e14 overflow | suv_ar → rf1 | Pass |
| tc04 | Visitor level closed, dry weather | visitor and family bays out of service | car_sd → rf1 | Pass |
| tc05 | Visitor level closed and raining | k8 adds 300 m to the roof | car_sd → dr1, car_gh → dr2 | Pass |
| tc06 | Saad's EV is charged | s2 no longer fires; k4 +40 m on chargers | ev_sa → oc1 | Pass |
| tc07 | Saad off call, EV charged | no on-call entitlement | ev_sa → st3, suv_ar → st4 | Pass |
| tc08 | Waqar's ban is lifted | NAF in a3 succeeds | car_wq → vs1 | Pass |
| tc09 | Yasir's plate re-read at 93% | s1 holds; outside visiting hours | car_ya → dr1; overstay(car_ya) | Pass |
| tc10 | Bushra's badge is renewed | s4 holds, a1 clears her | car_bu → st4 | Pass |
| tc11 | Accessible bays closed; Noor's van too tall | f1 fails for L1 (230 > 210 cm) | wv_no → dr1, car_ks → dr2, car_gh → dr2 | Pass |
| tc12 | Delivery van misses its slot | t4 fails, a4 does not clear | van_ju denied: outside_delivery_slot | Pass |
| tc13 | Tahir arrives on time | t2 holds, patient zone opens | car_th → pt2 | Pass |
| tc14 | Kashif visits in the evening | t3 holds in 17:00–20:00 | car_ks → vs1 | Pass |
| tc15 | Dr Areeba two hours early | t1 fails: only drop-off | suv_ar → dr1; ks, gh, th → dr2 | Pass |
| tc16 | Unknown car blocks an ambulance bay | w3, w4 with unregistered (NAF) | amb_fa → amb2; tow_request(amb1) | Pass |
| tc17 | Both ambulance bays blocked | priority 0 falls back to drop-off | amb_fa → dr1; two tow requests | Pass |
| tc18 | Two ambulances at 09:00 | same priority, arrival order decides | amb_rz → amb2 | Pass |
| tc19 | Drop-off congestion | waiting queue, retried on departure | car_ks waits 15 min | Pass |
| tc20 | Car with two classes | IC6; DL clash SUV / PetrolCar | IC6 reported | Pass |
| tc21 | Doctor recorded as a visitor | IC13 | IC13 reported | Pass |
| tc22 | Badge expires before issue | IC9 | IC9 reported | Pass |

### 8.1.3 Inference accuracy and consistency

- All 22 scenarios give the expected decisions, alerts and constraint results in Prolog (`swipl main.pl scenarios`: 22/22 pass).
- The JavaScript engine reproduces every Prolog result: decisions, alerts, fees, constraint witnesses, fact counts and cycle counts for all 22 scenarios (71 tests).
- The forward and backward chainers agree: a test proves backward every admitted/1 fact the forward chainer derives, and fails backward on every refused vehicle.
- The DL and rule views agree: the AdmittedVehicle and ClearedPerson concepts contain exactly the individuals the rules admit and clear.
- All 21 FOL statements beyond Horn hold on the baseline model; the three bad-data scenarios are caught by IC6, IC13 and IC9.
- The baseline day runs in under a second: 431 facts, 857 derived, 13 cycles, 1,300 facts at the fixed point.

## 8.2 Discussion

### 8.2.1 Whether the objectives were met

**Table 34: Objectives against results**

| Objective | Result |
|---|---|
| Frames, script, semantic network | 30 class + 16 instance frames with 7 facet kinds and 6 demons; 10-scene script; 61-node, 79-link network |
| FOL and DL | 36 FOL statements (21 beyond Horn, all model-checked); 89 DL axioms with realisation and consistency |
| Horn clauses, forward and backward chaining | 97 rules in 3 strata; full forward and backward traces with proof trees and why-not |
| Safe negation | Stratified negation as failure, checked automatically |
| Constraints and tests | 13 integrity constraints; 22 scenarios; 159 + 71 automated tests, all passing |
| Interactive explanation | 3D digital twin and reasoning bench running the same rules in the browser |

### 8.2.2 Expressivity against tractability

**Propositional logic** has no variables, so every rule must be written once per individual. "A visitor who is not banned is cleared" becomes VisitorWaqar ∧ ¬BannedWaqar → ClearedWaqar, repeated for all 18 people. Candidate(v,b) over 18 vehicles and 29 bays needs 522 atoms, and cost(v,b,c) with every possible cost needs thousands more; adding one bay means rewriting every rule that mentions bays. Times can only be compared with an atom for every minute. Its gain is tractability: satisfiability is decidable (NP-complete in general) and Horn propositional logic is decided in linear time.

**First-order logic** removes those limits: one rule ∀p (Visitor(p) ∧ ¬Banned(p) → Cleared(p)) covers every visitor, quantifiers state "exactly one driver" and "no cheaper free bay", and comparisons handle time and cost. The price is that entailment in full FOL is only semi-decidable: a prover may run forever on a query that does not follow. FOL is also monotonic, so it cannot say "assume not banned unless told otherwise", and it has no built-in defaults.

**Table 35: Expressivity and tractability of the formalisms**

| | Propositional | Full FOL | DL | Horn + NAF (used) |
|---|---|---|---|---|
| Rules for "visitor not banned → cleared" | 18 (one per person) | 1 | 1 definition | 1 |
| Candidate bays | 522 atoms | 1 predicate | 1 role | 1 predicate |
| Time and cost arithmetic | one atom per value | yes | concrete domains only | builtins on bound values |
| "Every person has some role" (disjunction) | yes, ground | yes | yes (covering axiom) | no: checked after the run |

| | Propositional | Full FOL | DL | Horn + NAF (used) |
|---|---|---|---|---|
| Rules joining several individuals (cost from zone and department) | ground only | yes | no | yes |
| Defaults and exceptions | no | no (monotonic) | no | NAF, closed world |
| Decidability | decidable; NP-complete (Horn: linear) | semi-decidable | decidable; ExpTime for this expressivity | decidable; polynomial |

CarePark-KR therefore decides with stratified Horn clauses, the most expressive of these that still answers in polynomial time, and uses FOL and DL to specify and check. What the Horn restriction costs is paid for explicitly: disjunctions (f3) and optimality (f24) are checked on the finished model; uniqueness (f4, f5) and non-overlap (f23) are integrity constraints; negation is negation as failure, sound only under the closed-world assumption, which suits a gate (an unlisted ban does not exist) but not open data. The rules have no function symbols that build new terms and builtins run only on bound values, so forward chaining always reaches a fixed point.

### 8.2.3 Forward against backward chaining

**Table 36: Forward and backward chaining compared**

| | Forward chaining | Backward chaining |
|---|---|---|
| Starts from | the 431 facts | one goal, e.g. admitted(car_bu) |
| Work done | derives all 857 facts, most never asked about | only the facts the goal needs (19 calls for admitted(wv_no)) |
| Best for | deciding the whole day at the gate; monitoring | checking or explaining one decision |
| Negation | safe through stratification | safe through NAF on ground goals |
| Explanation | the firing log shows when each fact appeared | proof tree and why-not tree show why |
| Termination | guaranteed (finite fixed point) | depth limit 40 |

### 8.2.4 Limitations observed

- Bay choice is greedy in time order: each vehicle gets the cheapest bay free when it arrives, which is not always the best for the whole day (a later patient may find the patient zone taken by an earlier one).
- The closed-world assumption means a missing fact is read as false; a ban entered late would not stop a vehicle already admitted.
- Walking distances are a fixed table; the system does not model lifts that are out of order or blocked corridors.
- The DL reasoner is a structural checker for this ontology, not a full tableau reasoner.
- Times are whole minutes within one day; overnight shifts across midnight would need dates on every time.

# 9. CONCLUSION

## 9.1 Conclusion and Future Improvements

CarePark-KR shows that a hospital parking policy can be captured as explicit knowledge and run with explanations. The same knowledge is represented five ways: frames and a script for concepts and procedure, a semantic network for relations, FOL for the exact policy, DL for the vocabulary and its consistency, and 97 Horn clauses that the system executes. Forward chaining decides a whole day of 15 arrivals in 13 cycles; backward chaining proves or refutes any single decision and names the condition that failed. All 22 test scenarios, including code red, rain, height limits, blocked ambulance bays and bad data, give the expected results, and the 3D digital twin lets anyone watch the reasoning happen.

Future improvements:

- Replace the greedy bay choice with a look-ahead that reserves patient and maternity bays for appointments due in the next hour.
- Read live occupancy sensors and ANPR events instead of a scripted day.
- Add probabilistic reasoning for plate reads between 70% and 90% instead of a hard threshold.
- Export the ontology to OWL and check it with a full DL reasoner.
- Add dates to times so overnight shifts and multi-day stays are handled.

# 10. REFERENCES


1. Brachman, R. J. and Levesque, H. J. Knowledge Representation and Reasoning. Morgan Kaufmann, 2004.
2. Russell, S. and Norvig, P. Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020. Chapters 7–10 (logic, inference, knowledge representation).
3. Baader, F., Calvanese, D., McGuinness, D., Nardi, D. and Patel-Schneider, P. (eds.) The Description Logic Handbook, 2nd ed. Cambridge University Press, 2007.
4. Minsky, M. A Framework for Representing Knowledge. MIT AI Laboratory Memo 306, 1974.
5. Schank, R. C. and Abelson, R. P. Scripts, Plans, Goals and Understanding. Lawrence Erlbaum, 1977.
6. Apt, K. R., Blair, H. A. and Walker, A. Towards a theory of declarative knowledge. In Foundations of Deductive Databases and Logic Programming, Morgan Kaufmann, 1988 (stratified negation).
7. Clark, K. L. Negation as Failure. In Logic and Data Bases, Plenum Press, 1978.
8. Sterling, L. and Shapiro, E. The Art of Prolog, 2nd ed. MIT Press, 1994 (meta-interpreters).
9. Wielemaker, J. et al. SWI-Prolog. Theory and Practice of Logic Programming 12(1–2), 2012. https://www.swi-prolog.org
10. three.js documentation. https://threejs.org/docs


# 11. TEAM PARTICIPATION

**Table 37: Team participation**

| S. No. | Name | Roll No. | Contribution |
|---|---|---|---|
| 1 | Minhaj Khan | AI-23301 | Problem statement, knowledge acquisition, domain requirements, facts, frames, demons and the PARKING-VISIT script (sections 1–3.1) |
| 2 | Faraz Khan | AI-23047 | Semantic network, description logic TBox and ABox, realisation and consistency, expressivity vs tractability analysis (sections 3.2, 4.2, 8.2.2) |
| 3 | Raied Khan | AI-23046 | FOL formalisation, Horn clause conversion, rules and integrity constraints, forward chainer and day replay (sections 4.1, 5, 6.1) |
| 4 | Sabih Salam | AI-23040 | Backward chainer and why-not explainer, 3D website, JavaScript engine port, testing, report (sections 6.2, 7, 8.1) |